CS 530 Lab Notebook

Lab 9

Name: Jarren Calizo
OdinID: calizo
Date: November 26, 2025

Table of Contents

Lab Overview 5
Lab 9.1g OAuth2 Guestbook 6
Step 1. Motivation for OAuth2 Guestbook 6
Step 2. Authorization Code Flow 6
Step 3. Checking out the OAuth Guestbook 6
Step 4. OAuth initiation in sign.py 7
Step 5. Identity provider interaction 7
Step 6. Wiring the callback route 7
Step 7. Handling the callback and fetching the token 8
Step 8. Signing page with real identity 8
Step 9. Model and index template updates 9
Step 10. Building and deploying to Cloud Run 9
Step 11. OAuth consent screen setup 10
Step 12. Creating the OAuth client ID 11
Step 13. Updating Cloud Run environment variables 12
Step 14. Visit the application 12
1. Take a screenshot of the Headers that includes the URL and the returned HTTP status code for the first two requests for your lab notebook. 12
2. Based on the description of the source code, what lines of code in our application are responsible for the second request shown? 14
3. Take a screenshot of the permissions you (as a user) are granting the Guestbook access to. 15
4. Take a screenshot of the Headers that includes the entire Callback URL and its returned HTTP status code. What is the URI for the Location that the User sent to by the Callback? 15
5. Find the request within Developer Tools that fetches the embedded image and take a screenshot of its URL. 16
Step 15. Secrets manager deployment 19
1. Take a screenshot showing multiple authenticated accounts have been able to sign the Guestbook. 20
Step 16. Removing access 20
1. Take a screenshot of the expanded information that includes your OdinId for your lab notebook. 21
Step 17. Cleanup 21
Lab 9.2g Firebase 22
Step 1. Firebase Web Application 22
Step 2. Project Setup 22

Step 3. Application Setup 22
Step 4. Authentication setup 22
1. What other domains are given access to this Firebase project by default? 23
Step 5. Database Setup 24
Step 6. Storage Setup 24
Step 7. CLI Setup 24
Step 8. Bundling with Webpack 24
1. Take a screenshot of the first 10 lines of the produced file. 25
Step 9. Configure Firebase Within Application 25
Step 10. Initialize Firebase Within Application 25
Step 11. Test Application 25
Step 12. Add authentication 26
1. What missing functions deal with user authentication? 26
2. What missing functions deal with sending and receiving messages? 27
Step 13. Update UI 27
1. What are the names of the elements that are hidden when the user is signed out? 28
2. What is the name of the element that is not hidden when the user is signed out? 28
Step 14. Testing the Application with Authentication 29
Step 15. Adding Text Messaging 29
Step 16. Test application with text messaging 29
1. Include a screenshot of the message and its fields in the database for your lab notebook 30
Step 17. Manual message insertion 30
1. Include a screenshot of the application with its two messages for your lab notebook 30
Step 18. Add image messaging 31
1. What is the URL of the image that is first shown in the UI as the message is loading? 31
Step 19. Test application with image messaging 31
1. How do the fields in an image document differ from that of the text document? 32
2. What URL and storage location can the image be found at? 32
3. Take a screenshot of the image in the storage bucket for your lab notebook. 33
Step 20. Deploy application 33
1. What directory is the application going to be served from? 33

2. Take a screenshot of the message including the URL for your lab notebook. 34
Step 21. Clean Up 34
Lab 9.3g BigQuery, BigLake 35
Step 1. Enabling BigQuery Services 35
Step 2. Examining the Dataset 35
Step 3. Create dataset 35
1. Take a screenshot of the table's details that includes the number of rows in the table. 36
Step 4. Query data 36
1. Screenshot the query results and include it in your lab notebook 37
2. Screenshot your results and include it in your lab notebook 38
3. Screenshot your results and include it in your lab notebook 38
4. Screenshot your results and include it in your lab notebook 39
Step 5. Beginning the BigLake Component 39
Step 6. Creating the External Table (BigLake) 39
Step 7. Configuring the BigLake Connection 39
Step 8. Granting Storage Permissions to the Connector 39
Step 9. Query data 40
1. Screenshot the query results and include it in your lab notebook 40
Lab 9.4g Jupyter Notebooks 41
Step 1. Notebooks Lab 1 (Natality) 41
Step 2. Launch notebook 41
Step 3. BigQuery query 41
1. How much less data does this query process compared to the size of the table? 42
2. How many twins were born during this time range? 43
3. How much lighter on average are they compared to single babies? 43
Step 4. Jupyter notebook query 44
Step 5. Exploring the dataset 44
Step 6. Run queries 44
1. Show the plots generated for the two most important features for your lab notebook 45
Step 7. Notebooks Lab 2 (COVID-19 data) 46
Step 8. Mobility 46
1. What day saw the largest spike in trips to grocery and pharmacy stores? 46
2. On the day the stay-at-home order took effect (3/23/2020), what was the total impact on workplace trips? 46
Step 9. Airport traffic 46

1. Which three airports were impacted the most in April 2020 (the month when lockdowns became widespread)? 47
2. Run the query again using the month of August 2020. Which three airports were impacted the most? 48
Step 10. Mortality 49
1. What table and columns identify the place name, the starting date, and the number of excess deaths from COVID-19? 49
2. What table and columns identify the date, county, and deaths from COVID-19? 50
3. What table and columns identify the date, state, and confirmed cases of COVID-19? 50
4. What table and columns identify a county code and the percentage of its residents that report they always wear masks? 51
Step 11. Run example queries 51
1. Show a screenshot of the plot and the code used to generate it for your lab notebook 51
2. From within your Jupyter notebook, run the query and write code that shows the first 10 states that reached 1000 deaths from COVID-19. Take a screenshot for your lab notebook. 52
3. Take a screenshot for your lab notebook of the Top 5 counties and the states they are located in. 53
Step 12. Write queries 53
1. Plot the results and take a screenshot for your lab notebook. 53
2. Plot the results and take a screenshot for your lab notebook. 54
Step 13. Cleanup 55

Lab Overview

Lab 9 felt like stepping into the living ecosystem of modern cloud systems and watching each layer breathe. Across OAuth2, Firebase, BigQuery, BigLake, and Jupyter Notebooks, the lab wove together the identity, data, and analytics layers that quietly anchor real-world applications. It was the kind of lab that reminds me why Internet, Web, and Cloud Systems is so foundational: every component speaks a different dialect of the same language of distributed design.

The journey began with the OAuth2 Guestbook, where trust becomes a first-class citizen. Instead of accepting whatever name a user claims, the Guestbook leaned into Google as the identity provider, shifting authentication into a formal handshake. From the 302-step dance between /sign, Google’s login, and the /callback endpoint, to the moment my profile photo materialized inside the signing page, I could see the Authorization Code flow running exactly as the textbooks promise. Integrating Secrets Manager forced a deeper appreciation for operational hygiene. Removing the CLIENT_SECRET from environment variables and watching the service break, then resurrecting it through secure secret injection, made the lifecycle of identity-driven apps feel real. Even revoking access at the end completed a necessary emotional arc: systems are not only built, but also responsibly unwound.

Firebase carried a different kind of energy. If OAuth2 felt like choreography, Firebase felt like momentum. FriendlyChat came alive the moment authentication wired in correctly and the real-time database began echoing messages across clients. Each step - bundling with webpack, initializing Firebase, filling in empty helper functions, securing auth-gated UI flows, posting text, uploading images, and deploying to Hosting - unfolded like watching a single-page app discover its own heartbeat. It was empowering to build a system where the client is not a spectator but an active participant in authentication, storage, and real-time messaging.

BigQuery and BigLake shifted the focus again, this time from systems to data. Uploading the newborn-names dataset, querying top names through the console, CLI, and interactive shell, and comparing native vs external tables offered a grounded view into how scalable analytics actually works. The symmetry between querying warehouse-backed tables and lake-backed external files made the platform feel both structured and fluid at once. BigQuery’s ease of slicing massive datasets never failed to surprise me.

The final chapter - Jupyter Notebooks on Vertex AI - pulled everything together through analysis and narrative. Running optimized natality queries, quantifying cost reductions by shrinking scan sizes, and visualizing predictors like plurality and gestation weeks made data feel alive. Then turning toward COVID-19 public datasets brought a weight that went beyond SQL: plotting early mobility spikes, watching airport traffic collapse, tracing excess deaths, and mapping statewide timelines reminded me that behind each row is a story. The notebook became not just a tool, but a lens.

In the end, Lab 9 was more than a sequence of tasks. It was a walk through the modern cloud stack - identity, frontend, storage, real-time sync, warehouse analytics, data lakes, and exploratory notebooks - each piece reinforcing why cloud systems matter and how they interlock. It felt like both a technical milestone and an emotional one, stitching together the abstractions that power the world we build.

Lab 9.1g OAuth2 Guestbook

Step 1. Motivation for OAuth2 Guestbook

I started by reviewing the lab description and reminding myself why this version of the Guestbook matters: instead of trusting whatever name and email a user types, we lean on Google as an identity provider. The goal is simple but powerful - only authenticated users can sign, and their entries are tied to real Google account metadata (name, email, profile picture).

Step 2. Authorization Code Flow

Next I walked through the Authorization Code flow on paper and mapped each step to the Guestbook routes:

- User clicks the "Sign" link on the index page.
- /sign checks for an OAuth token. If missing, it starts the OAuth dance with Google.
- Google shows the login + consent screen.
- After successful login and consent, Google redirects to /callback with a one time code and state.
- /callback exchanges that code for an access token and then redirects the user back to /sign.
- With a valid token, /sign fetches userinfo and renders the signing form with real identity attached.

This set the mental model before touching any code.

Step 3. Checking out the OAuth Guestbook

In Cloud Shell I pulled down the lab code and switched into the OAuth version:

```
cd ~
git clone https://github.com/wu4f/cs430-src
cd cs430-src/09_oauth
```

A quick ls and tree -L 2 confirmed the expected structure:

- app.py, index.py, sign.py, callback.py, logout.py, oauth_config.py
- gbmodel/ with __init__.py, Model.py, model_datastore.py
- templates/index.html and templates/sign.html

Step 4. OAuth initiation in sign.py

I opened oauth_config.py and sign.py and verified that the configuration is driven entirely by environment variables:

```
client_id = os.environ.get('CLIENT_ID')
client_secret = os.environ.get('CLIENT_SECRET')
redirect_callback = os.environ.get('REDIRECT_CALLBACK')
authorization_base_url = 'https://accounts.google.com/o/oauth2/auth'
token_url = 'https://accounts.google.com/o/oauth2/token'
```

In sign.py, the Sign view checks for an OAuth token. If there is no token, it creates an OAuth2Session with the redirect URI and the two scopes userinfo.email and userinfo.profile, then asks Google for an authorization URL and redirects the user there. The screenshot of sign.py shows the exact block that builds the OAuth2Session, sets the scope, and calls authorization_url(...) before return redirect(authorization_url).

Step 5. Identity provider interaction

Conceptually, this step is where Google forces authentication and then shows the consent screen. I did not change code here. Instead I made sure I understood that:

- The redirect URI is the Cloud Run URL with /callback.
- Google sends back state, code, and scope as query parameters on that callback.
- The state value is used as a CSRF defense and must match what was stored before redirect.

The actual interaction is captured later in the DevTools screenshots.

Step 6. Wiring the callback route

In app.py I confirmed that the callback route is registered:

```
from callback import Callback

app.add_url_rule(
   '/callback',
   view_func=Callback.as_view('callback'),
   methods=['GET']
)
```

No edits were needed, but this check makes it clear where the OAuth redirect lands inside the Flask application.

Step 7. Handling the callback and fetching the token

In callback.py I walked through the core logic:

- Re create the OAuth2Session with client_id, redirect_callback, and the stored session['oauth_state'].
- Force HTTPS in the callback URL by replacing http: with https:.
- Call fetch_token(token_url, client_secret=client_secret, authorization_response=request.url) to exchange the one time code for an access token.
- Store the resulting token in session['oauth_token'].
- Redirect back to the signing page via return redirect(url_for('sign')).

The DevTools screenshot of the /callback request later shows the 302 redirect and the Location: /sign header that this line produces.

Step 8. Signing page with real identity

Back in sign.py, the get handler for Sign uses the saved token:

```
if 'oauth_token' in session:
    google = OAuth2Session(client_id, token=session['oauth_token'])
    userinfo =
google.get('https://www.googleapis.com/oauth2/v3/userinfo').json()
    return render_template(
        'sign.html',
        name=userinfo['name'],
        email=userinfo['email'],
        picture=userinfo['picture']
    )
```

templates/sign.html then:

- Shows the profile picture via <img src="{{ picture }}" ...>
- Displays the name and email on the page

- Stores name, email, and picture as hidden fields in the form so they are included on the POST.

The screenshot of the Sign Guestbook page confirms that the lab is pulling my real name, email, and profile image from Google.

On POST, sign.py uses these fields to insert a new entry into the datastore and then redirects back to the index page.

Step 9. Model and index template updates

I confirmed that gbmodel/model_datastore.py adds a profile (or picture) field when creating an OAuthReview entity:

```
rev.update({
    'name': name,
    'email': email,
    'date': datetime.today(),
    'message': message,
    'profile': profile
})
```

templates/index.html has been updated to render a small <img> element in front of each entry based on this stored URL. The final Guestbook screenshot shows an entry where my profile picture, name, email, timestamp, and message are all displayed.

Step 10. Building and deploying to Cloud Run

I edited the Dockerfile to set my PSU email address as the maintainer, then used Cloud Build to build the container:

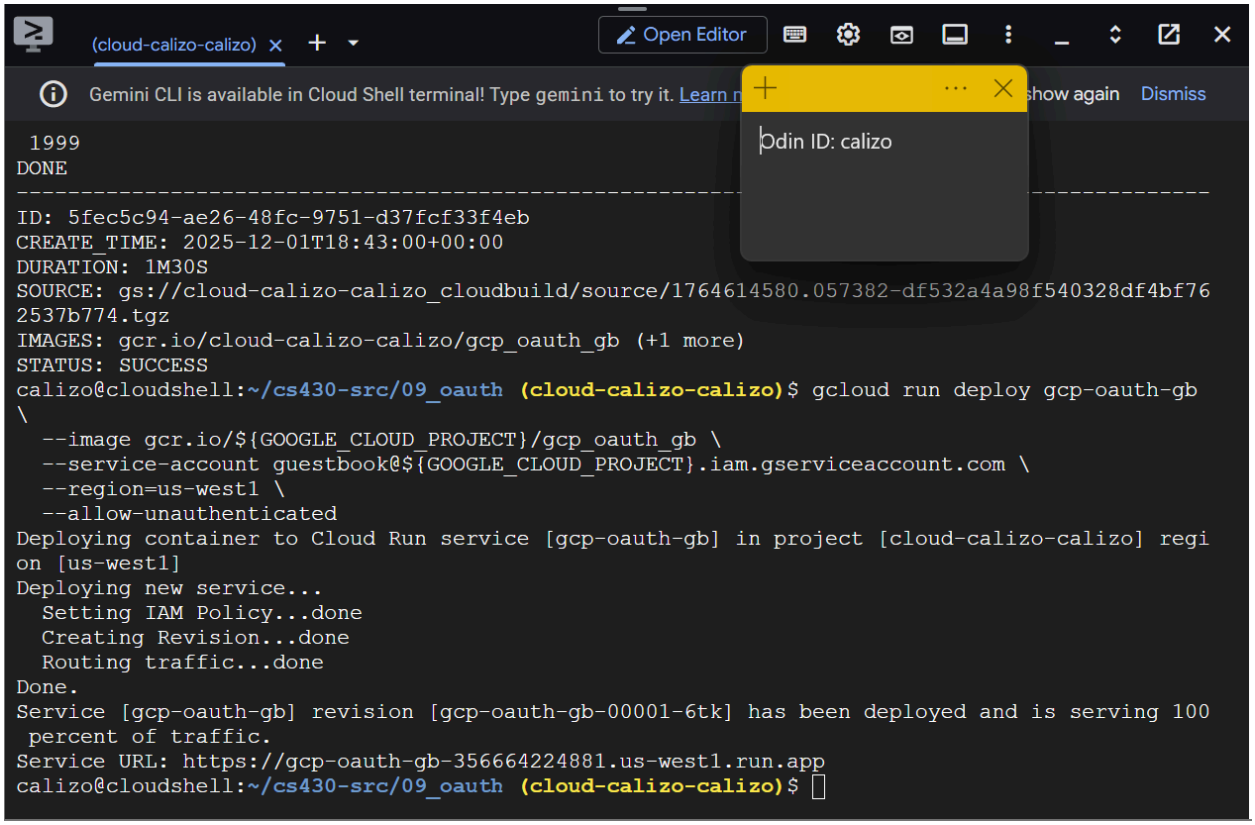
```
gcloud builds submit --timeout=900 \
  --tag gcr.io/${GOOGLE_CLOUD_PROJECT}/gcp_oauth_gb
```

After that I deployed to Cloud Run:

```
gcloud run deploy gcp-oauth-gb \
  --image gcr.io/${GOOGLE_CLOUD_PROJECT}/gcp_oauth_gb \
  --service-account
guestbook@${GOOGLE_CLOUD_PROJECT}.iam.gserviceaccount.com \
  --region=us-west1 \
  --allow-unauthenticated
```

The first screenshot shows a successful deployment and reports the service URL:

- https://gcp-oauth-gb-356664224881.us-west1.run.app



This URL becomes both the base of my application and the prefix for the callback endpoint.

Step 11. OAuth consent screen setup

In the Google Cloud console I configured the OAuth consent screen:

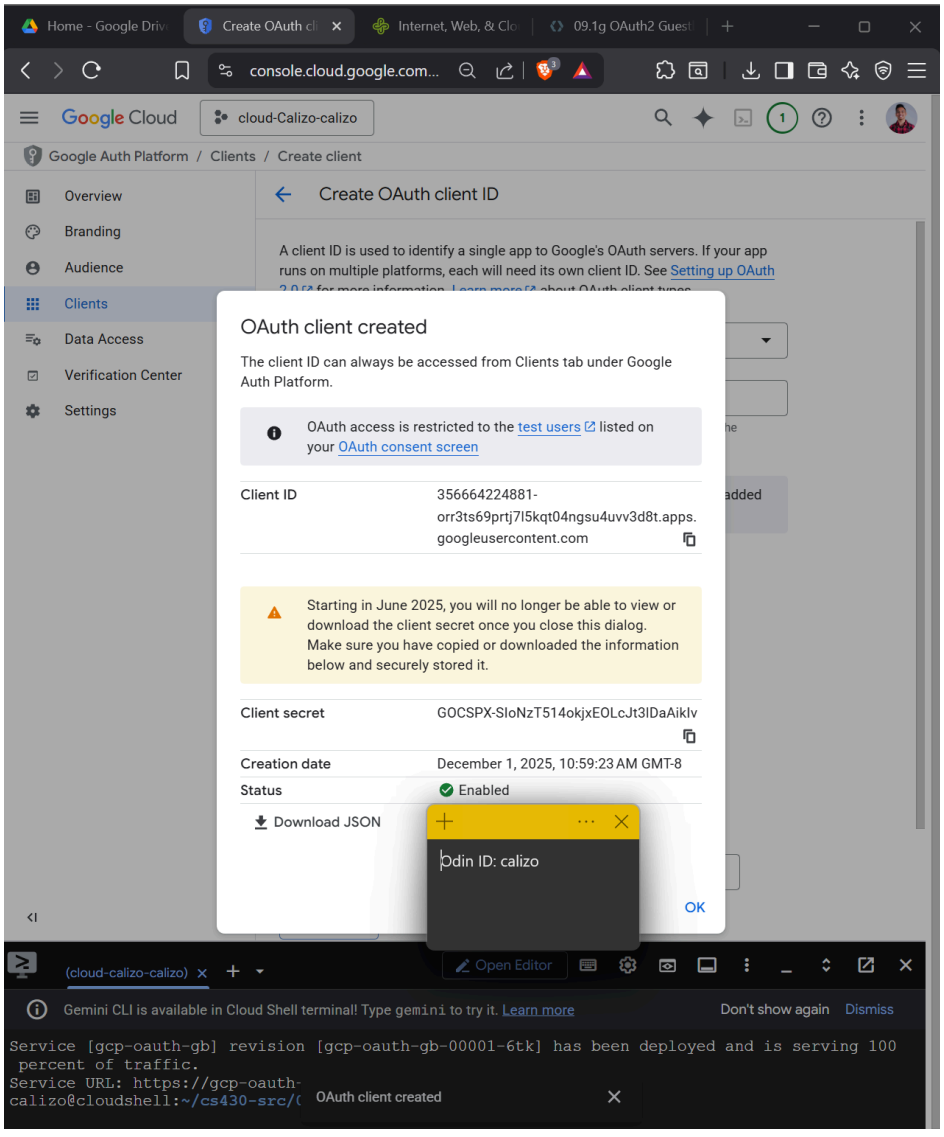
- User type: External
- App name and support email set to my pdx.edu identity
- Application home page: https://pdx.edu
- Authorized domain: pdx.edu

For scopes I enabled openid, which implicitly brings in userinfo.email and userinfo.profile. This aligns with the scopes requested in sign.py.

Step 12. Creating the OAuth client ID

Still in the console under APIs & Services -> Credentials I created a new OAuth client ID:

- Type: Web application
- Name: Guestbook
- Authorized redirect URI: https://gcp-oauth-gb-356664224881.us-west1.run.app/callback



The second screenshot captures the "OAuth client created" dialog, which includes the generated client ID and client secret. I recorded both values securely for use as environment variables.

Step 13. Updating Cloud Run environment variables

Back in Cloud Shell, I wired the OAuth configuration into the Cloud Run service via environment variables:

```
gcloud run services update gcp-oauth-gb \
  --update-env-vars
REDIRECT_CALLBACK=https://gcp-oauth-gb-356664224881.us-west1.run.app/ca
llback \
  --update-env-vars CLIENT_ID=<my-client-id> \
  --update-env-vars CLIENT_SECRET=<my-client-secret> \
  --region=us-west1
```

This connects the runtime configuration in oauth_config.py directly to the client registration I just created.

Step 14. Visit the application

Below I include each question prompt followed by my answer, tying it back to both the code and the captured screenshots.

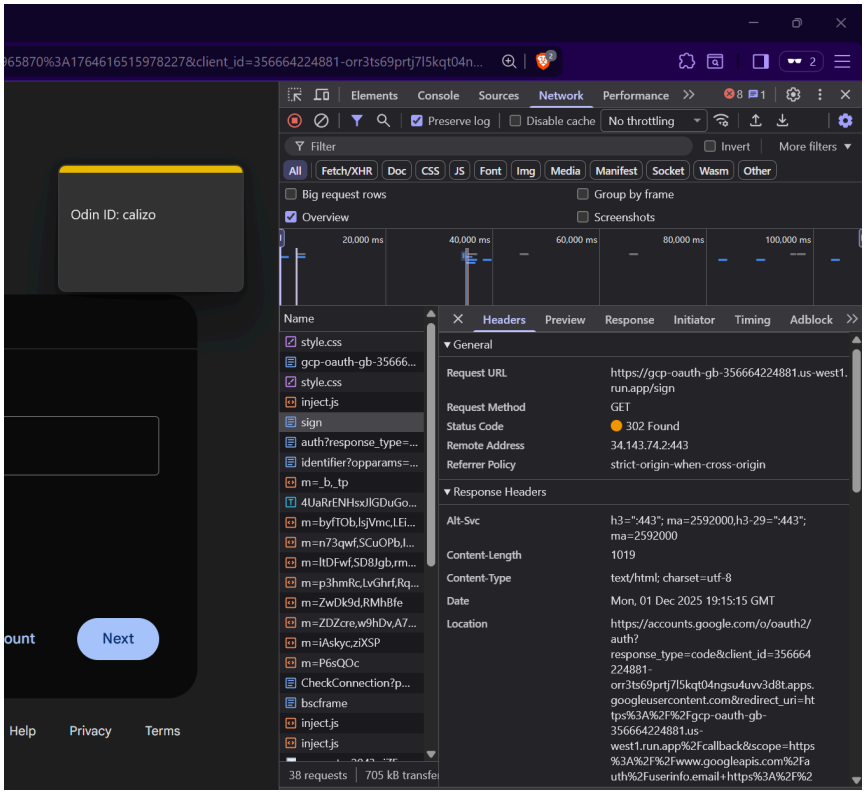
1. Take a screenshot of the Headers that includes the URL and the returned HTTP status code for the first two requests for your lab notebook.

For this step I:

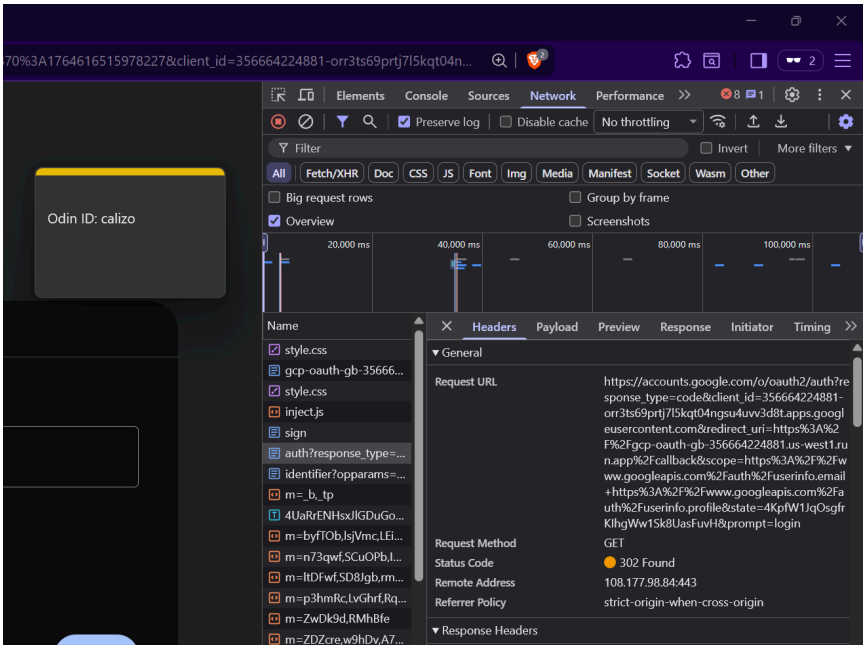
- Opened an Incognito window and navigated to the Cloud Run URL.
- Opened Developer Tools and switched to the Network tab with "Preserve log" enabled.
- Clicked the "Sign" link on the Guestbook index page.

The first two relevant requests are:

1. A GET request to the /sign endpoint at https://gcp-oauth-gb-356664224881.us-west1.run.app/sign
    - Status code: 302 Found
    - Response headers include a Location header pointing to a Google OAuth URL.

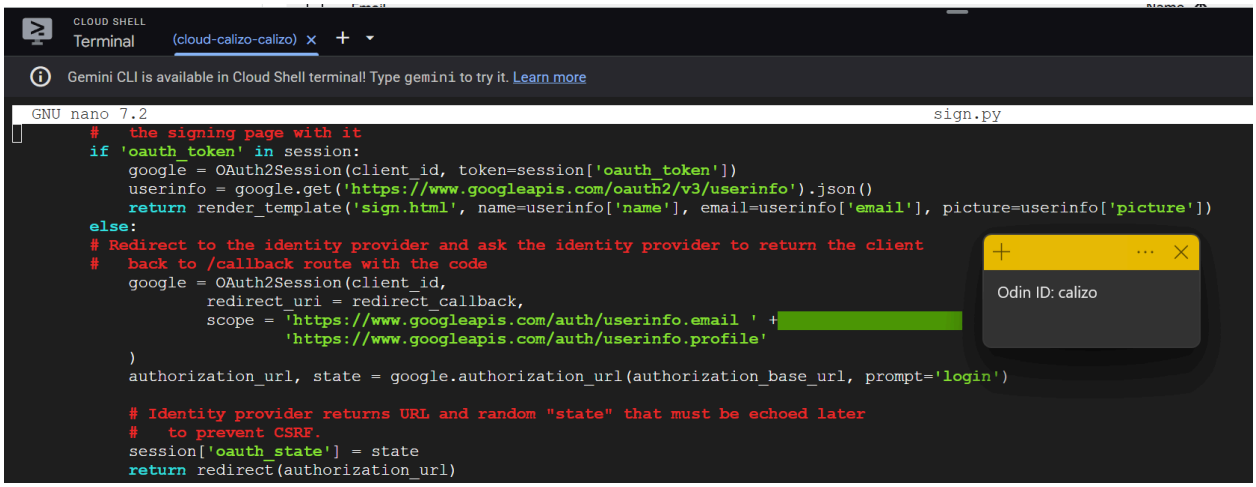


2. A GET request to the Google authorization endpoint at a long URL beginning with https://accounts.google.com/o/oauth2/auth?...
    - Status code: 302 Found



Both of these requests, along with their URLs and HTTP status codes, are captured in the first pair of Network tab screenshots.

2. Based on the description of the source code, what lines of code in our application are responsible for the second request shown?

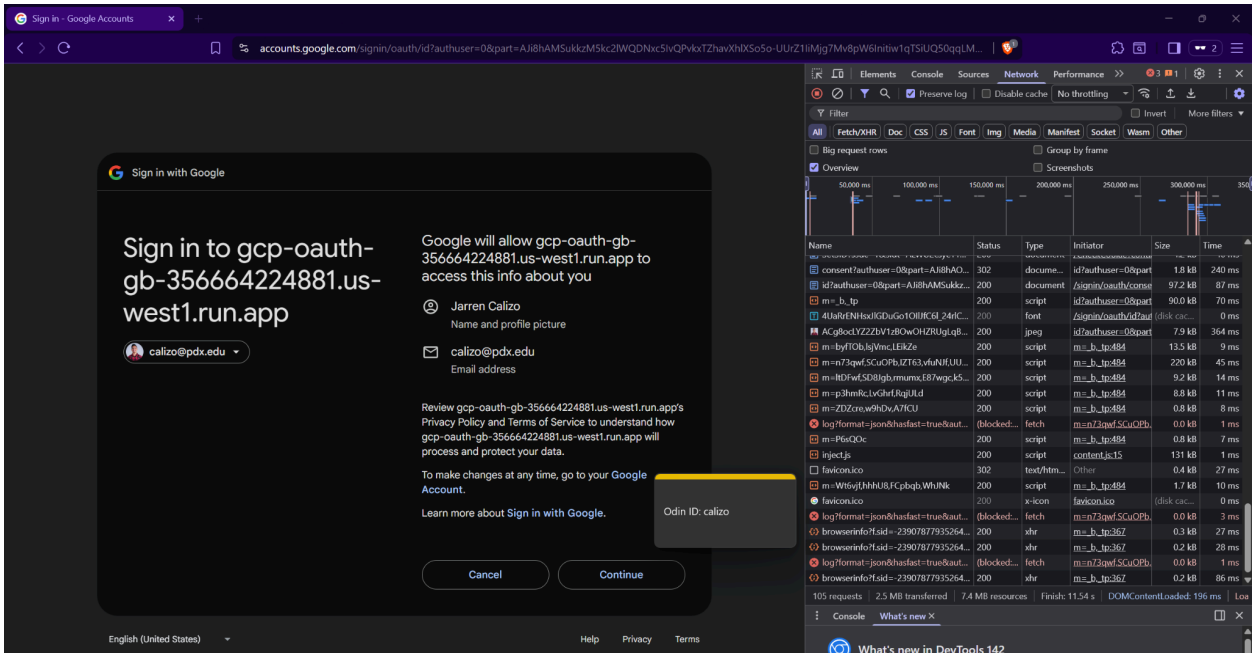


The second request is the browser being redirected to the Google authorization URL. In the Flask app, that behavior is triggered entirely inside the Sign view when no OAuth token is present. The relevant lines are in sign.py:

```
google = OAuth2Session(
    client_id,
    redirect_uri=redirect_callback,
    scope='https://www.googleapis.com/auth/userinfo.email ' +
        'https://www.googleapis.com/auth/userinfo.profile'
)
authorization_url, state = google.authorization_url(
    authorization_base_url,
    prompt='login'
)
session['oauth_state'] = state
return redirect(authorization_url)
```

authorization_url(...) constructs the Google URL, and return redirect(authorization_url) generates the HTTP 302 response that tells the browser to issue the second request to https://accounts.google.com/o/oauth2/auth?.... This exact block is visible in the sign.py screenshot.

3. Take a screenshot of the permissions you (as a user) are granting the Guestbook access to.



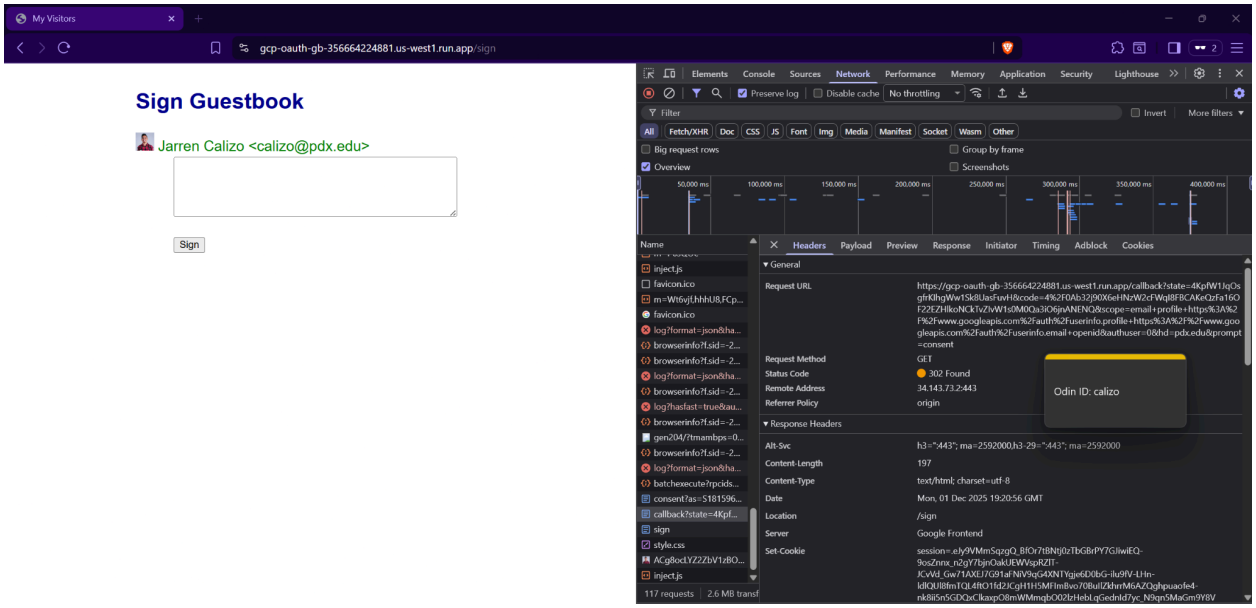
After following the redirect to Google and authenticating with my pdx.edu account, Google displayed the consent screen. The screenshot shows that the Guestbook app is being granted:

- Access to my name and profile picture
- Access to my email address

These correspond to the userinfo.profile and userinfo.email scopes requested by the application. The UI explicitly states that Google will allow the gcp-oauth-gb-356664224881.us-west1.run.app app to access "Name and profile picture" and "Email address."

4. Take a screenshot of the Headers that includes the entire Callback URL and its returned HTTP status code. What is the URI for the Location that the User sent to by the Callback?

After I clicked "Continue" on the consent screen, Google redirected back to my callback endpoint. In the Network tab I selected the request whose path starts with /callback:



- Request URL: https://gcp-oauth-gb-356664224881.us-west1.run.app/callback?state=...&code=...&scope=email+profile+openid
    - (the screenshot captures the full URL with state and code parameters)
- Status code: 302 Found

In the Response Headers for this /callback request, the Location header is:

- Location: /sign

Because this is a relative path, the user is effectively sent to:

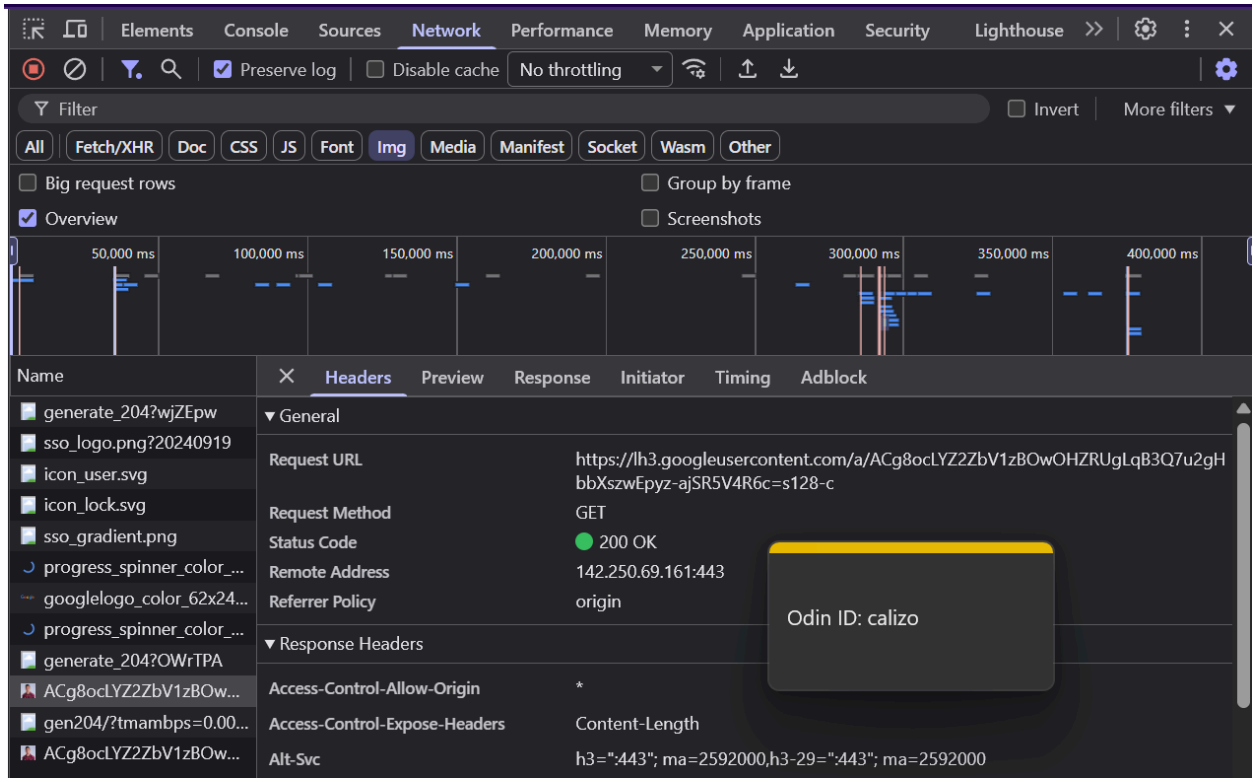
- https://gcp-oauth-gb-356664224881.us-west1.run.app/sign

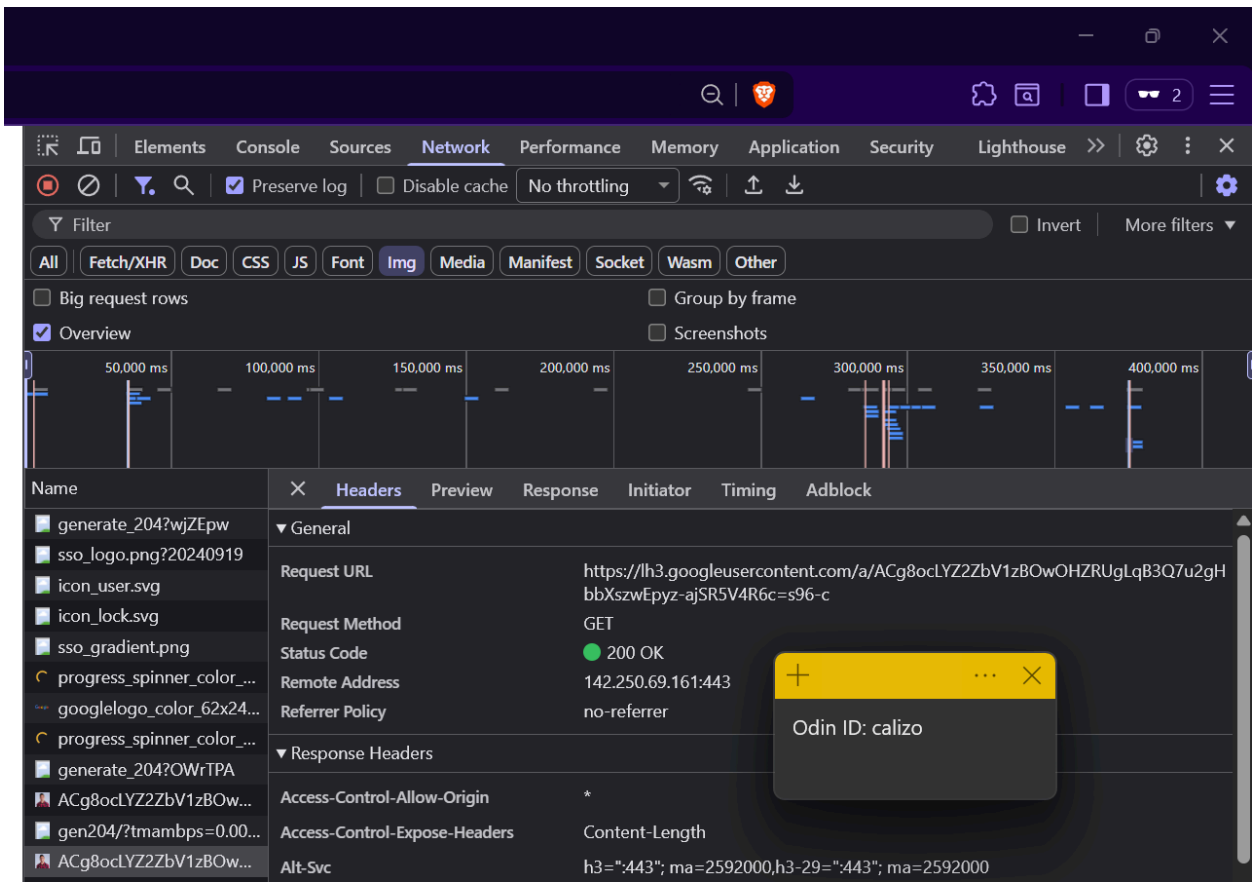
This redirect is produced by the final line of Callback.get() in callback.py:

- return redirect(url_for('sign'))

5. Find the request within Developer Tools that fetches the embedded image and take a screenshot of its URL.

Once the callback completed, the browser landed back on the Sign Guestbook page. The page includes my Google profile picture, so in the Network tab I filtered to Img requests and looked for the image used for that avatar.

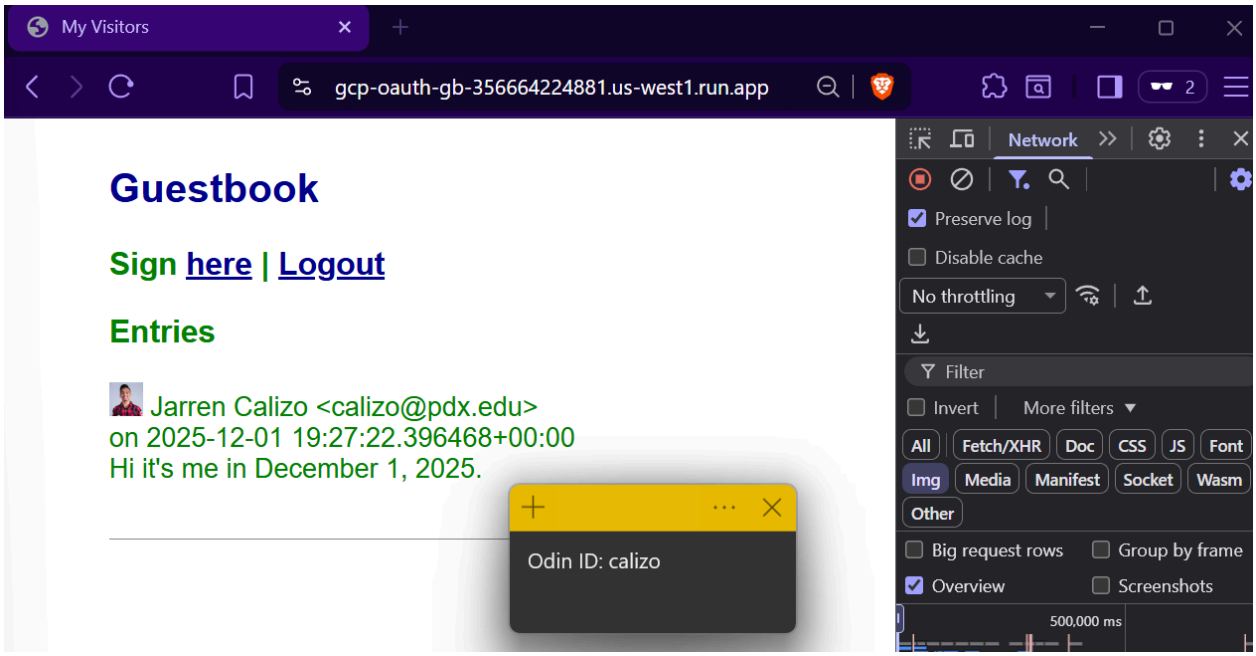




The relevant request has:

- Request URL starting with https://lh3.googleusercontent.com/a/ACg8ocLYZZZbV1zBOwO...
    - (the screenshot shows the full URL, including a size parameter such as =s128-c or =s96-c)
- Method: GET
- Status code: 200 OK

This is the embedded image that sign.html renders using the {{ picture }} field returned by the Google userinfo endpoint. The final Guestbook screenshot shows that this image is also rendered on the index page next to my entry, confirming that the URL was stored and reused correctly.



Insight

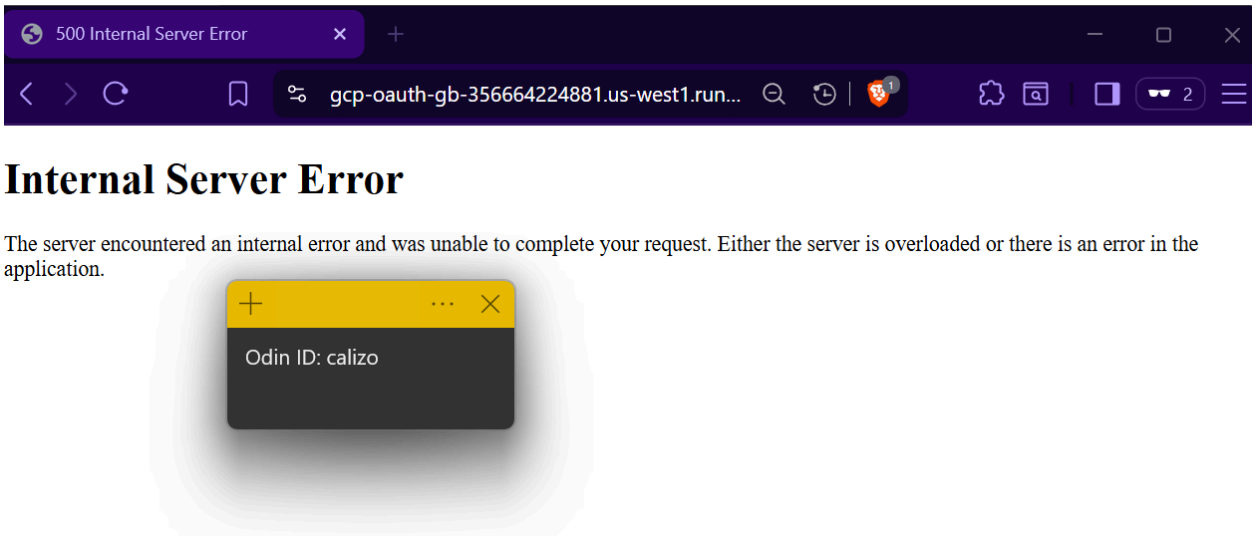
By the end of Step 14 I had the full lifecycle in motion:

- Container built and deployed to Cloud Run.
- OAuth consent screen and client credentials configured in GCP.
- Environment variables wired into oauth_config.py.
- DevTools confirming the Authorization Code flow hop by hop, with each 302 redirect aligning perfectly with the redirect(...) calls in sign.py and callback.py.
- A real Guestbook entry on the index page carrying my authenticated name, email, timestamp, message, and profile picture.

The screenshots serve as the concrete evidence of each of these checkpoints, while the code analysis ties the network traces back to the exact lines responsible for them.

Step 15. Secrets manager deployment

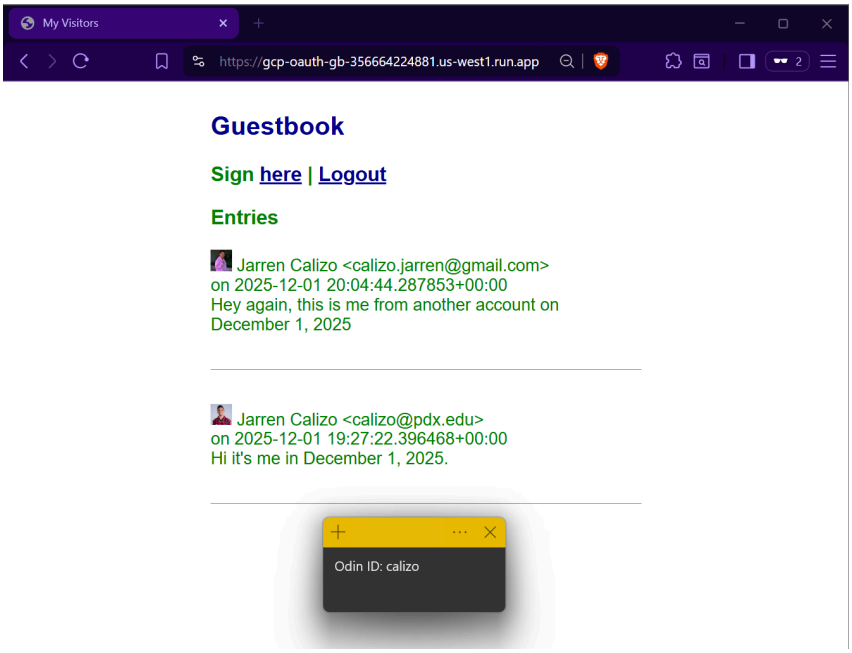
At this stage of the lab, I finally pushed the Guestbook into the more mature world of secret hygiene. Passing the CLIENT_SECRET through an environment variable was fine for development, but production asks for something sturdier. I removed the CLIENT_SECRET from the Cloud Run environment and, as expected, my next login attempt collapsed into a stark 500 error page. The screenshot captured this moment with a quiet clarity: a reminder that secure systems break loudly when a critical secret goes missing.



From there, I transitioned the CLIENT_SECRET into Secrets Manager. After storing the secret, granting the guestbook service account the secretAccessor role felt like handing a tightly controlled key to the right hands. Once Cloud Run was updated to source this secret securely, the service recovered gracefully.

To prove the full OAuth2 flow still worked, I authenticated again using a second Google account. The new entry appeared in the Guestbook, sitting right above my original pdx.edu post. This moment, shown in the screenshot, felt like a small victory two identities, two tokens, one securely functioning system.

1. Take a screenshot showing multiple authenticated accounts have been able to sign the Guestbook.



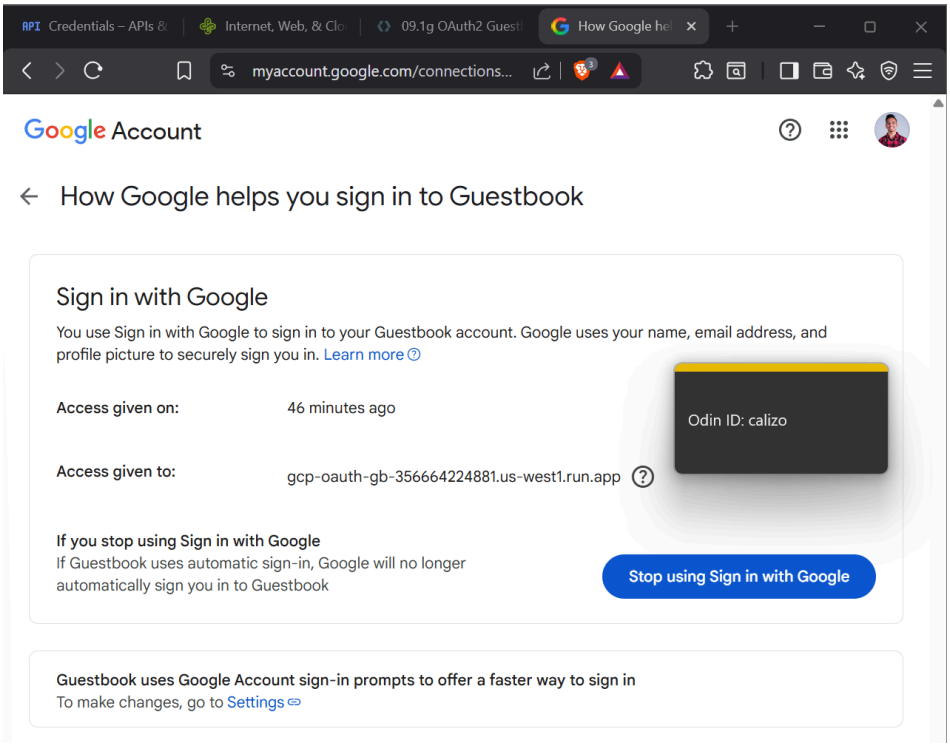
My evidence displayed the Guestbook with two unique entries: one from my pdx.edu account and another from my Gmail account. Both entries appear with distinct profile photos, email addresses, and timestamps, proving that the rebuilt OAuth flow works correctly after moving the client secret into Secrets Manager.

Step 16. Removing access

Once the application had proven itself, it was time to revoke the standing trust I had given it. OAuth teaches an important lesson: consent persists, and sometimes dangerously so. Navigating into my Google Account permissions, I located the Guestbook application under third-party access. Expanding it revealed the scope information and metadata tied to my identity. The screenshot captures this expanded view, showing the full access details granted earlier during sign-in.

After reviewing it, I removed access entirely. With one click, the Guestbook's persistent relationship to my Google identity was severed. This felt like closing the loop on the security lifecycle: build, grant, test, then revoke.

1. Take a screenshot of the expanded information that includes your OdinId for your lab notebook.



This shows the full detail view of the Guestbook OAuth entry: access time, the application identity, and the explanatory text Google displays when an app uses Sign in with Google. This satisfied the requirement to document the permissions I had previously granted.

Step 17. Cleanup

The final phase of the lab was a proper teardown. After confirming all functionality and screenshots, I deleted the Cloud Run service and its associated container image. Then, returning to the Google Cloud Console, I removed the OAuth client credentials entirely. Each deletion felt like sweeping away footprints from a journey, leaving the environment exactly as I found it.

With the service torn down, the container image erased, and the OAuth client removed, the Guestbook faded quietly back into the cloud. What remained was the lesson: the life of an OAuth-powered service is more than just the happy path. Secrets break things, access persists silently, and cleanup is part of responsible engineering.

This sequence of steps brought the whole story together: capability, control, and closure.

Lab 9.2g Firebase

Stepping into this lab felt like shifting gears from the OAuth2 Guestbook world into Firebase's client-first mindset. Unlike Cloud Run, where the server carries most of the responsibility, Firebase invites the browser to carry the rhythm: authentication, message passing, and realtime updates. What used to be an architecture of guarded entry points becomes a choreography of UI callbacks in a world where the client talks directly to the datastore.

In the first half, I built the FriendlyChat app from the ground up: initializing a fresh Firebase project, wiring authentication, linking Cloud Shell to the project, watching webpack breathe life into static HTML, and finally filling in the authentication helper functions that anchor the app’s identity flow.

Step 1. Firebase Web Application

Created the base Firebase project following the codelab pattern. This establishes the realtime database and storage backbone that FriendlyChat will rely on.

Step 2. Project Setup

I created a new project firebase-calizo, paired it with the course billing account, and mirrored it into the Firebase console. Analytics was turned off as instructed.

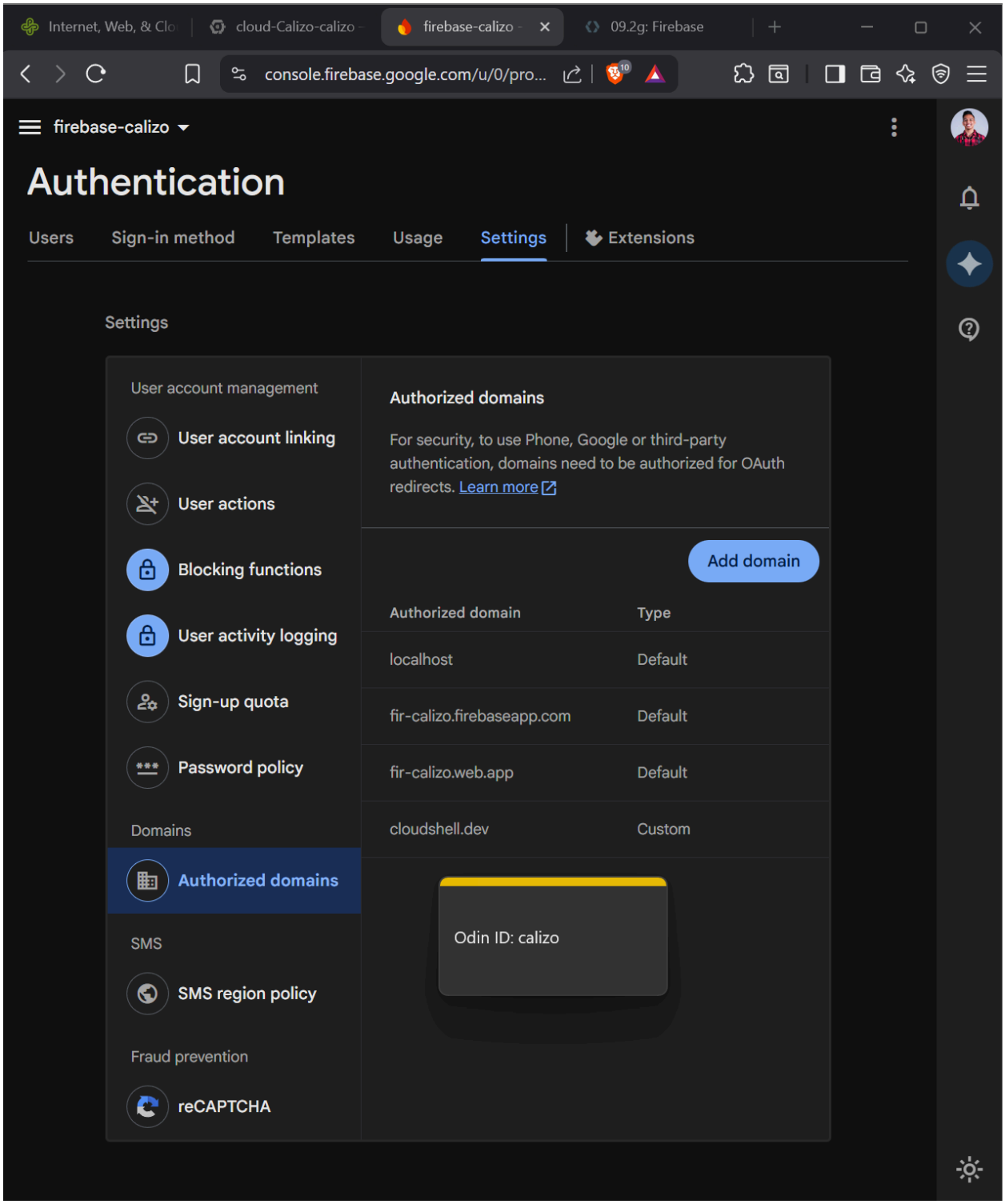
Step 3. Application Setup

Within the Firebase console, I registered a Web App for the project. I skipped the auto-integration steps because the Cloud Shell repository would handle resource initialization.

Step 4. Authentication setup

Enabled Google authentication under Build -> Authentication.
Authorized the necessary Cloud Shell domain to allow local development access.

1. What other domains are given access to this Firebase project by default?



The default authorized domains shown in the screenshot were:

- localhost
- fir-calizo.firebaseapp.com

- fir-calizo.web.app

These are automatically added to support local testing, hosting previews, and production hosting.

Step 5. Database Setup

Created a Firestore Database in test mode and kept the default region.

Step 6. Storage Setup

Enabled Firebase Storage using test mode rules and accepted the default bucket region.

Step 7. CLI Setup

Inside Cloud Shell, I cloned the FriendlyChat starter repository, installed Node 20 using nvm, and installed the Firebase CLI globally.

I performed:

```
firebase logout
firebase login --no-localhost
```

Then connected the CLI to my Firebase project with:

```
firebase use --add
```

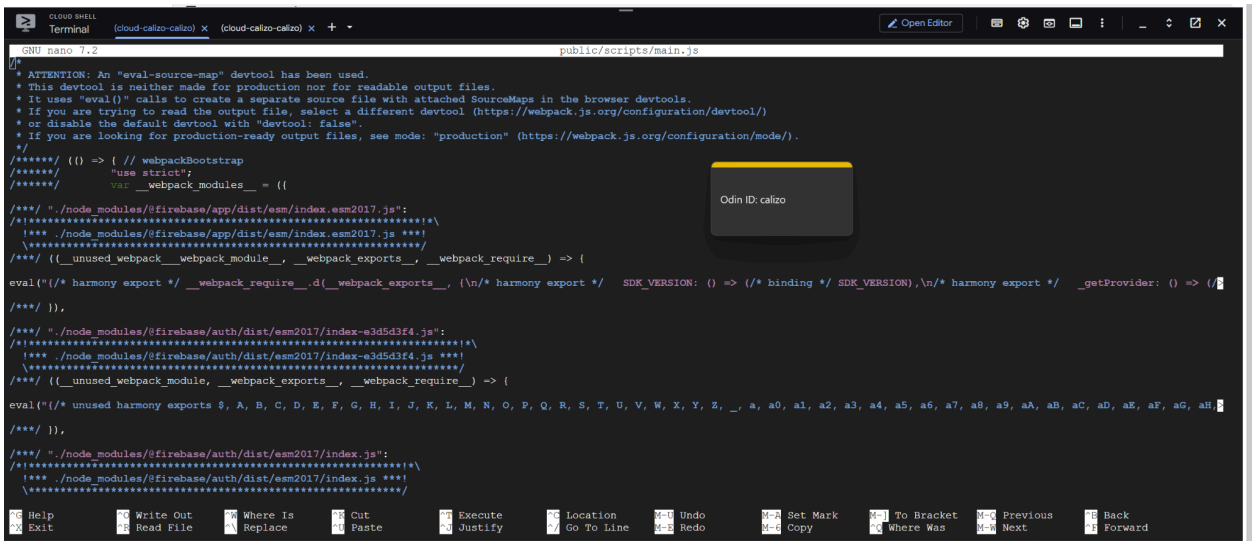
Alias was set to default.

Step 8. Bundling with Webpack

Webpack generated the public/scripts/main.js file after running:

```
npm install
npm run start
```

1. Take a screenshot of the first 10 lines of the produced file.



This captures webpack's autogenerated banner, confirming the build succeeded.

Step 9. Configure Firebase Within Application

I retrieved the firebaseConfig object from the Firebase console under Project Settings and pasted the values into:

- src/firebase-config.js

Webpack automatically rebuilt the bundle.

Step 10. Initialize Firebase Within Application

Added initialization logic to the end of src/index.js so that initializeApp(firebaseConfig) occurs before the rest of the application logic.

Step 11. Test Application

Launched the local Firebase Hosting emulator:

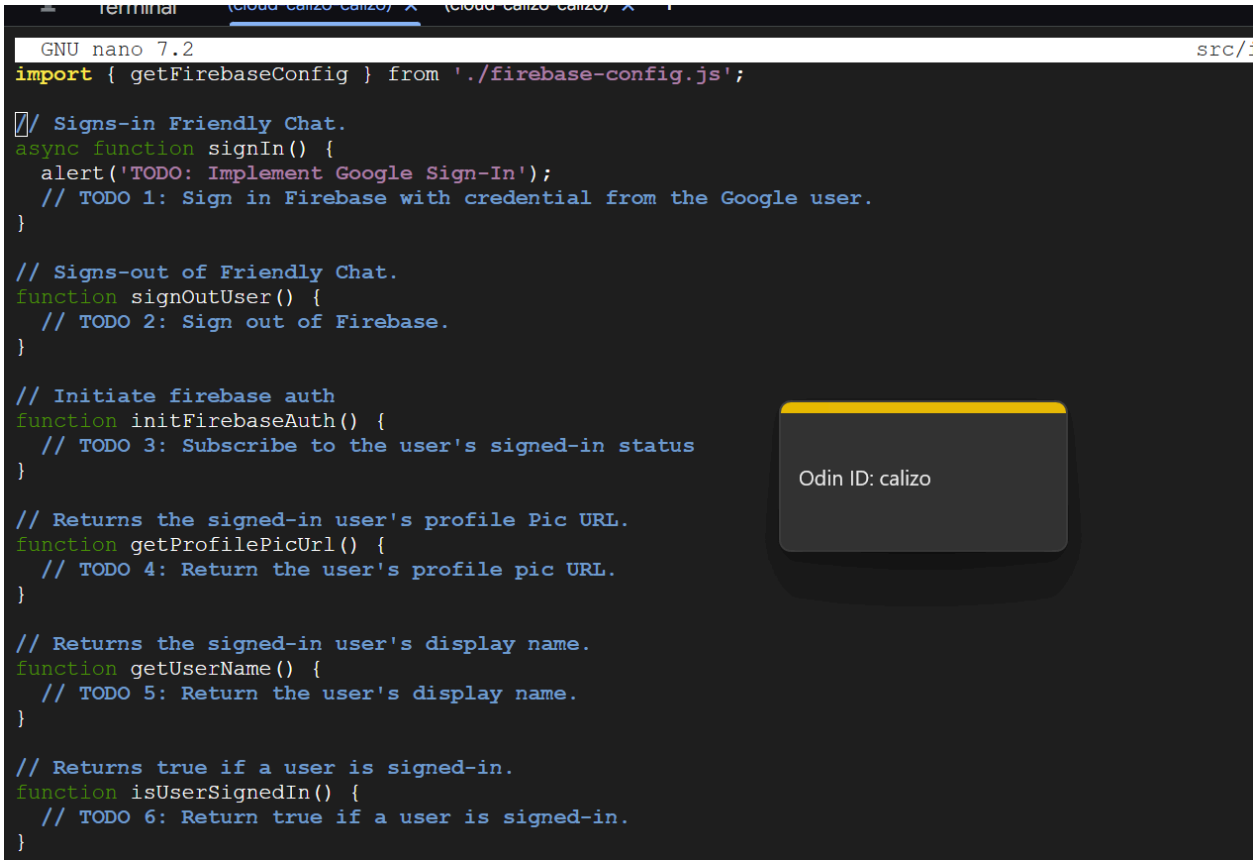
firebase serve --only hosting

Opened the preview on port 5000 and verified that the FriendlyChat UI skeleton loaded properly from the authorized domain. Then, I exited with Ctrl+C.

Step 12. Add authentication

Reviewed the incomplete helper functions in src/index.js.

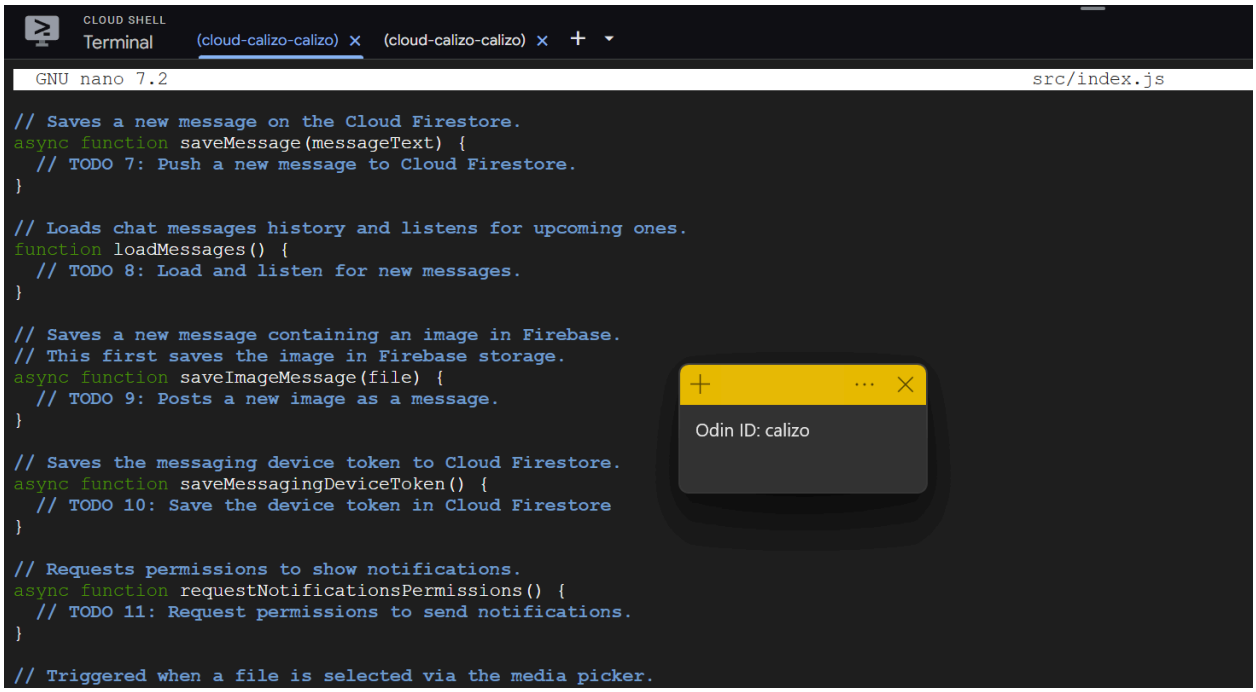
1. What missing functions deal with user authentication?



- signIn
- signOutUser
- initFirebaseAuth
- getProfilePicUrl
- getUserName
- isUserSignedIn

These handle login, logout, UI updates, and user metadata retrieval.

2. What missing functions deal with sending and receiving messages?



- saveMessage
- loadMessages
- saveImageMessage
- saveMessagingDeviceToken
- requestNotificationsPermissions

These control posting text, images, tokens, and preparing notifications.

I implemented the required authentication logic:

```
var provider = new GoogleAuthProvider();
await signInWithPopup(getAuth(), provider);
```

And for sign-out:

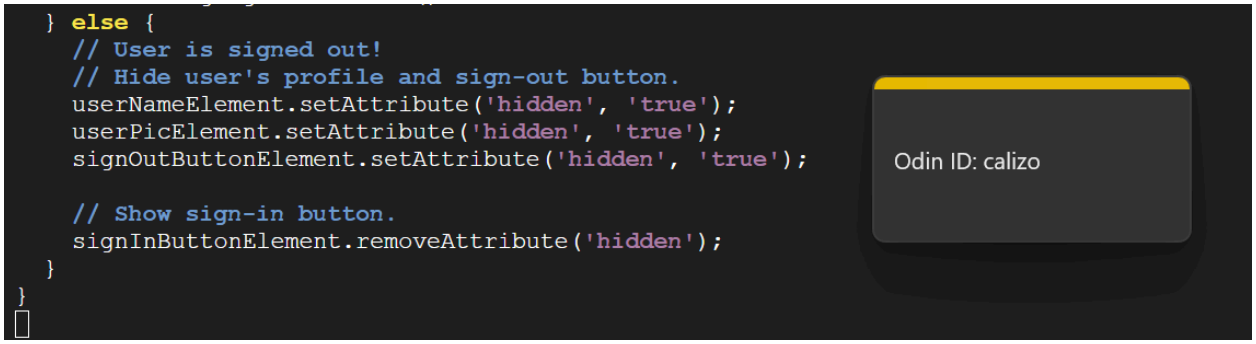
```
signOut(getAuth());
```

Step 13. Update UI

Connected the auth state listener:

```
onAuthStateChanged(getAuth(), authStateObserver);
```

This toggles visibility of elements depending on whether the user is authenticated.



1. What are the names of the elements that are hidden when the user is signed out?

Based on this screenshot above of the else block, the following elements are hidden:

- userNameElement
- userPicElement
- signOutButtonElement

2. What is the name of the element that is not hidden when the user is signed out?

- signInButtonElement

Implemented the corresponding functions to finalize auth behavior:

```
return getAuth().currentUser.photoURL || '/images/profile_placeholder.png';
return getAuth().currentUser.displayName;
return !!getAuth().currentUser;
```

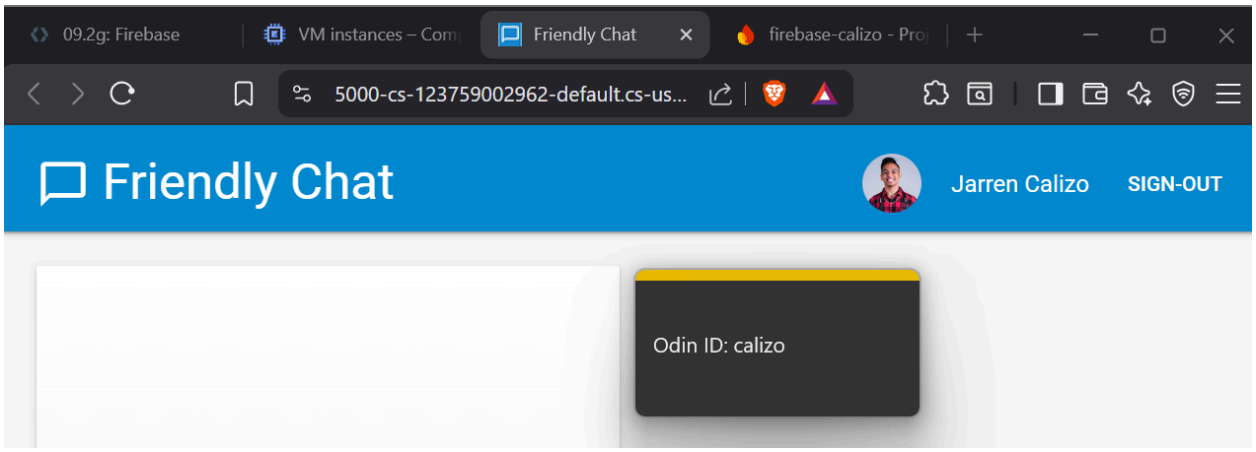
These complete the identity layer required for message posting later in the lab.

Insight

Stepping into the second half of the Firebase lab felt like watching the pieces of a modern cloud app finally come alive. Authentication, real-time sync, storage, deployment, and the arc that ties everything together all began to feel tangible the moment Friendly Chat stopped being a scaffold and started acting like an application with a heartbeat.

Step 14. Testing the Application with Authentication

I relaunched the Firebase hosting emulator from Cloud Shell and, as expected, the app immediately enforced authentication. Trying to send a message while signed out triggered the error dialog, confirming that my isUserSignedIn() logic worked as intended.



After clicking the "SIGN-IN WITH GOOGLE" button, the popup flow succeeded and the UI updated with my profile picture and name, as shown in the screenshot. This was the first moment the application felt personalized: Friendly Chat knew who I was.

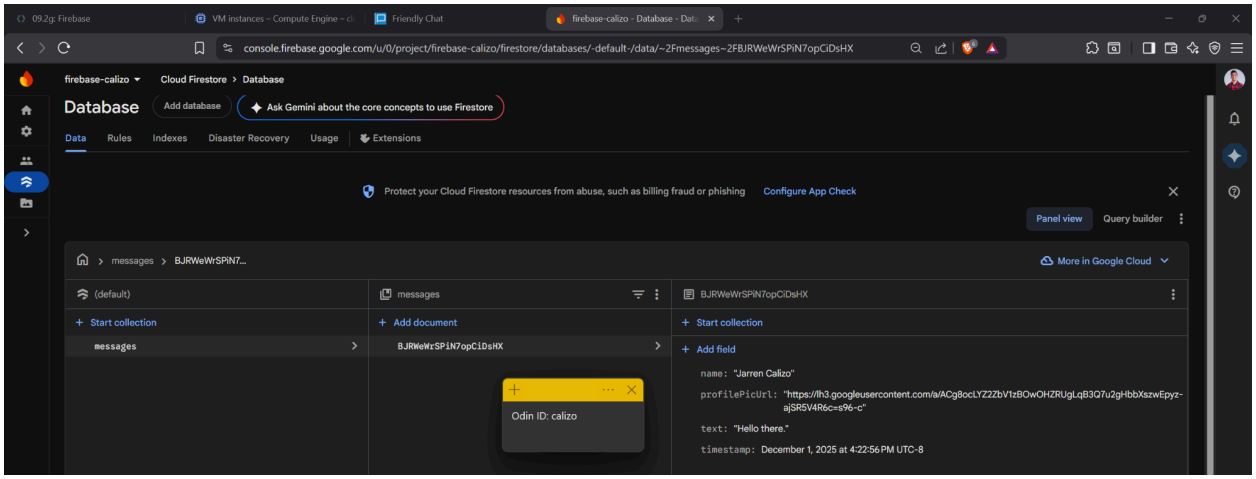
Step 15. Adding Text Messaging

I returned to src/index.js and completed the text message addDoc() implementation and real-time message listener with onSnapshot(). Once updated, the emulator reflected new chat messages instantly across all connected clients. The Firestore screenshot shows the message document created by sending "Hello there." through the UI.

Step 16. Test application with text messaging

After launching the emulator again, sending a message worked end-to-end: the chat updated immediately, and Firestore created the corresponding document in the messages collection. The screenshot shows the database entry with expected fields such as name, profilePicUrl, text, and timestamp.

1. Include a screenshot of the message and its fields in the database for your lab notebook

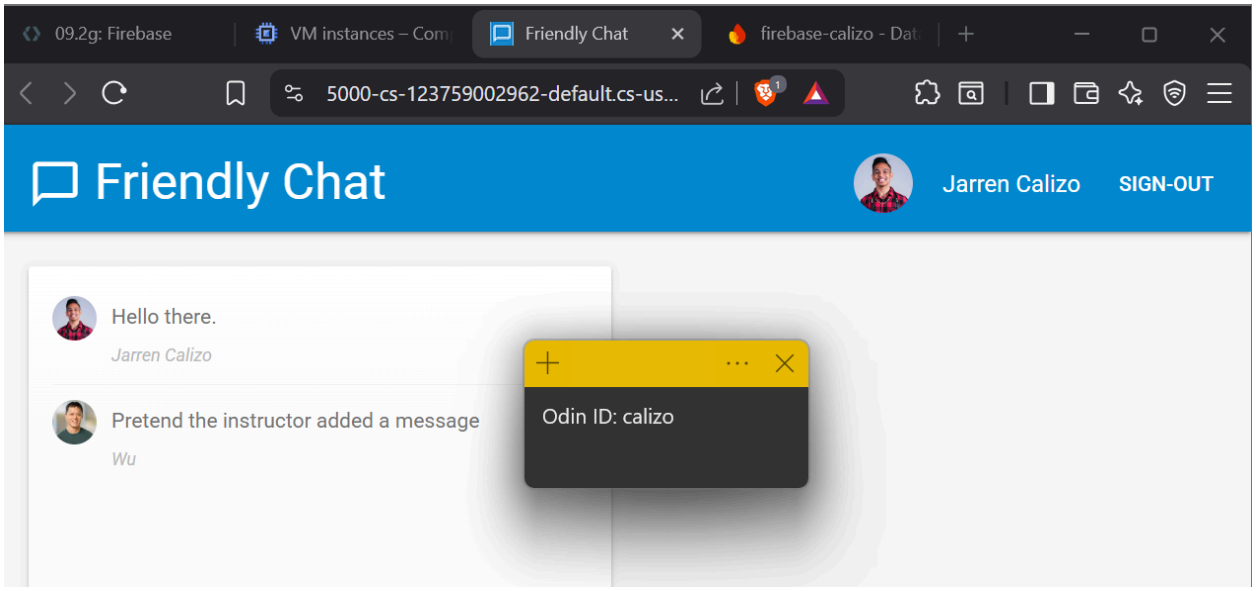


This displays the document in Firestore and the matching UI message in Friendly Chat.

Step 17. Manual message insertion

Next, I used the Firestore console to manually insert a new message with the instructor's information and a custom timestamp. When I returned to the app, the message appeared instantly beside my original one, validating the real-time listener behavior.

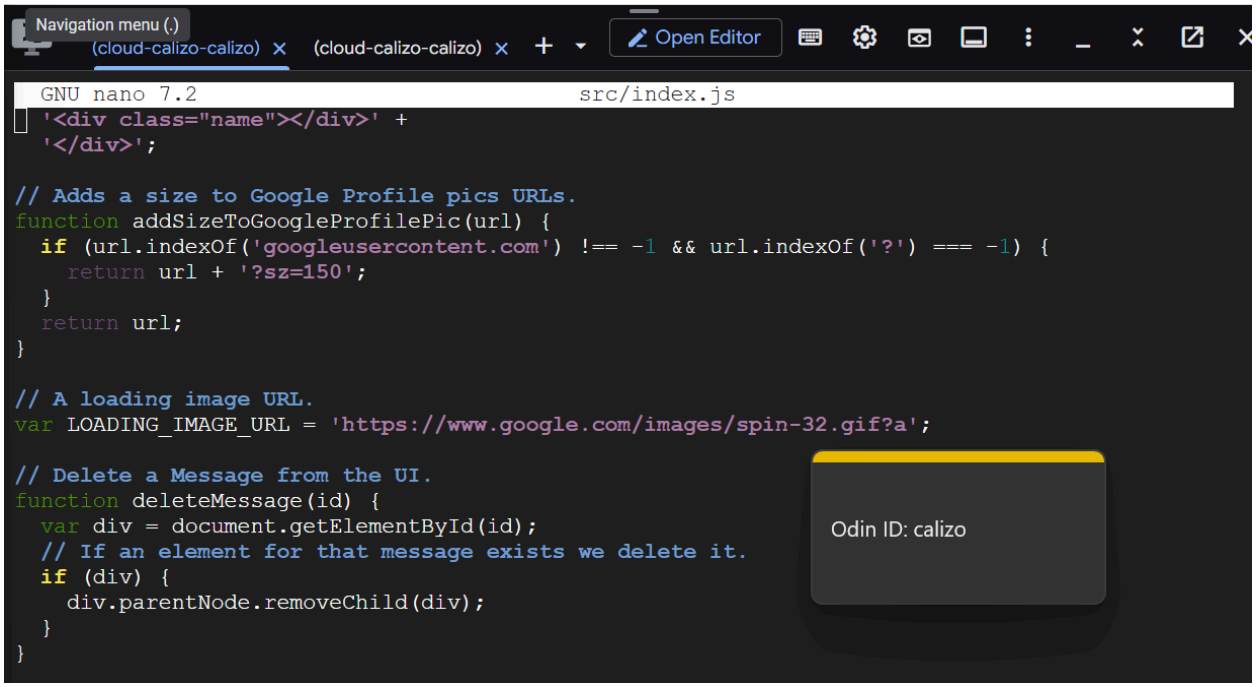
1. Include a screenshot of the application with its two messages for your lab notebook



Step 18. Add image messaging

I implemented the asynchronous image-upload path using Cloud Storage. This flow added a placeholder message, uploaded the file to the bucket, obtained its public URL, and updated the Firestore document accordingly. The code block defines LOADING_IMAGE_URL, which Firebase displays until the upload completes.

1. What is the URL of the image that is first shown in the UI as the message is loading?

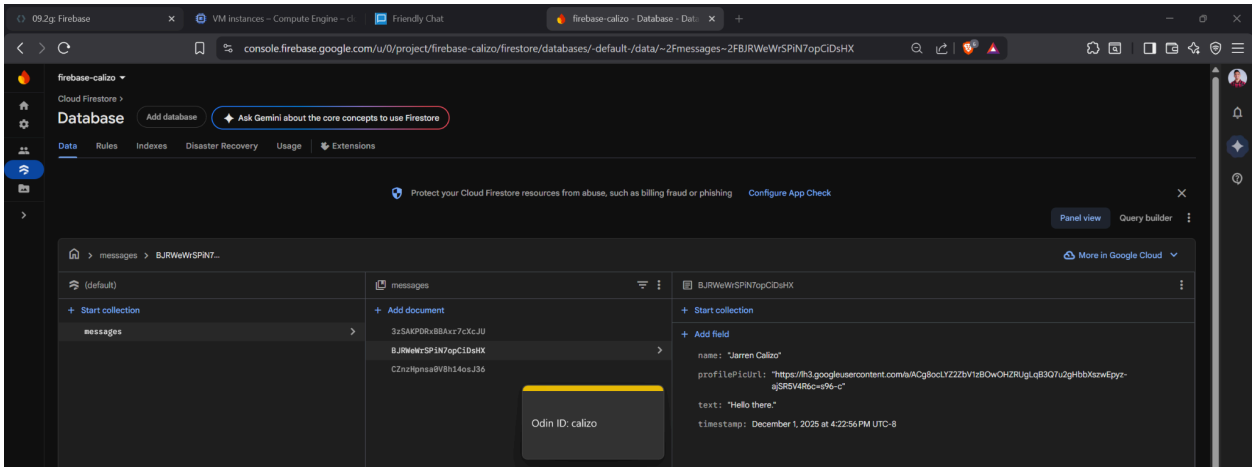


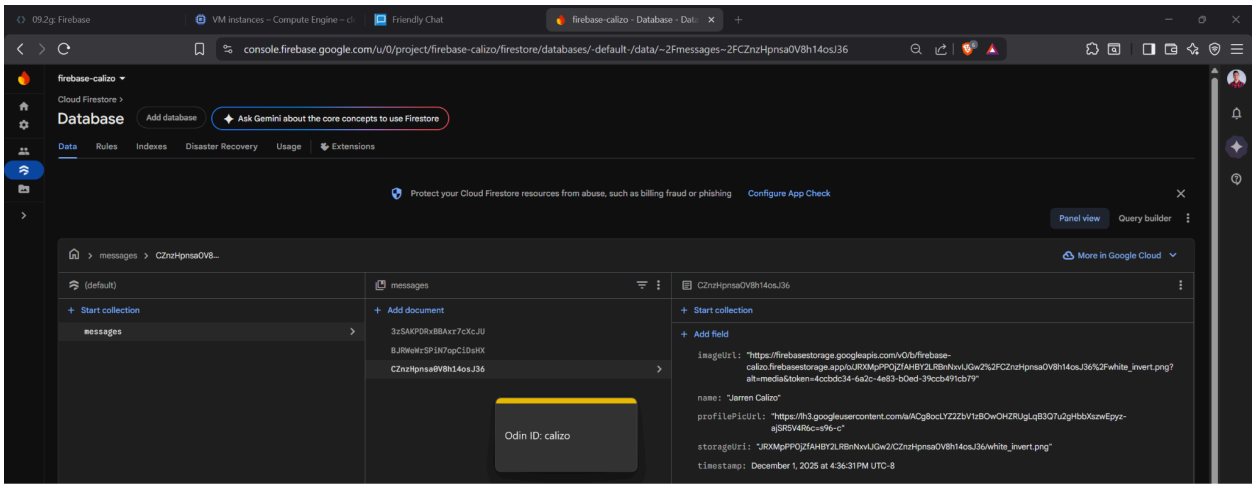
- https://www.google.com/images/spin-32.gif?a

Step 19. Test application with image messaging

After serving the application again and uploading an image, the UI updated exactly as expected: first a loading spinner, then the final image. In Firestore, the message document contains additional fields that distinguish it from a standard text message.

1. How do the fields in an image document differ from that of the text document?



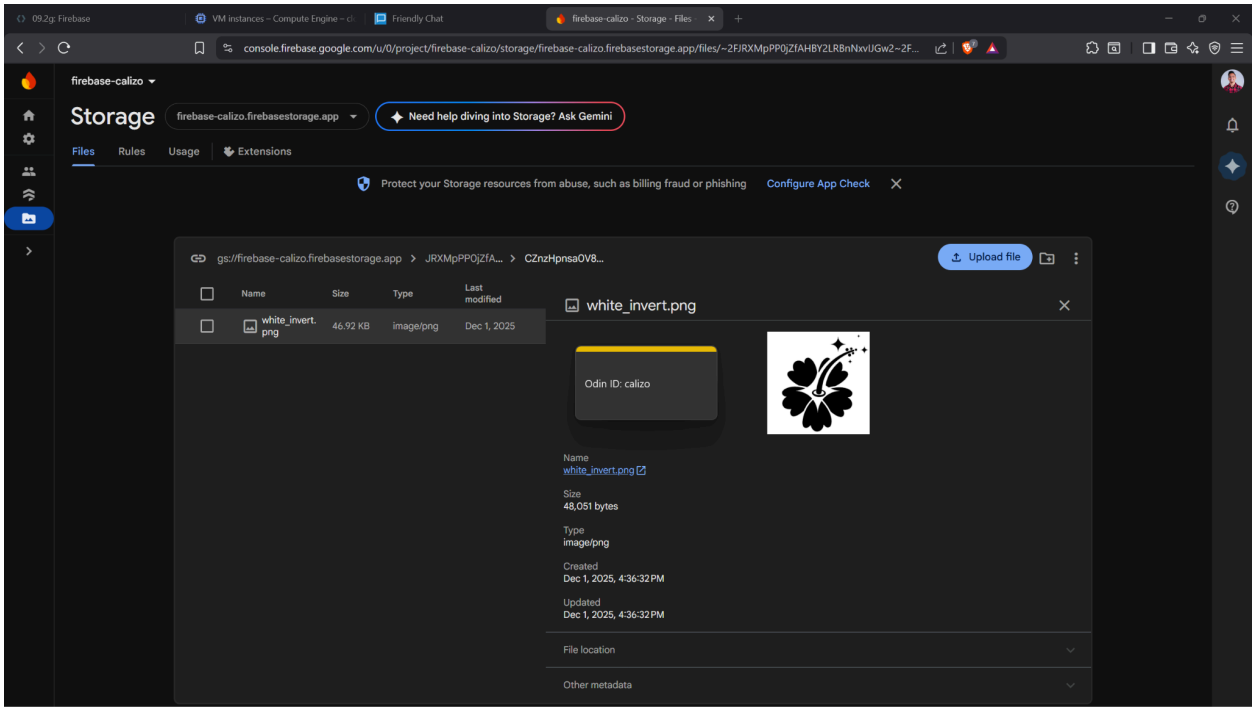


- It includes imageUrl instead of (or in addition to) text.
- It contains storageUri to record the file's storage location.
- The placeholder image loads first before being updated with the final public URL.

2. What URL and storage location can the image be found at?

In the screenshot above, this is showing the firebasestorage.googleapis.com URL and the Storage URI path formatted like:
<uid>/<docID>/<filename>

3. Take a screenshot of the image in the storage bucket for your lab notebook.



Step 20. Deploy application

I reviewed firebase.json to confirm the hosting configuration before deployment.

1. What directory is the application going to be served from?



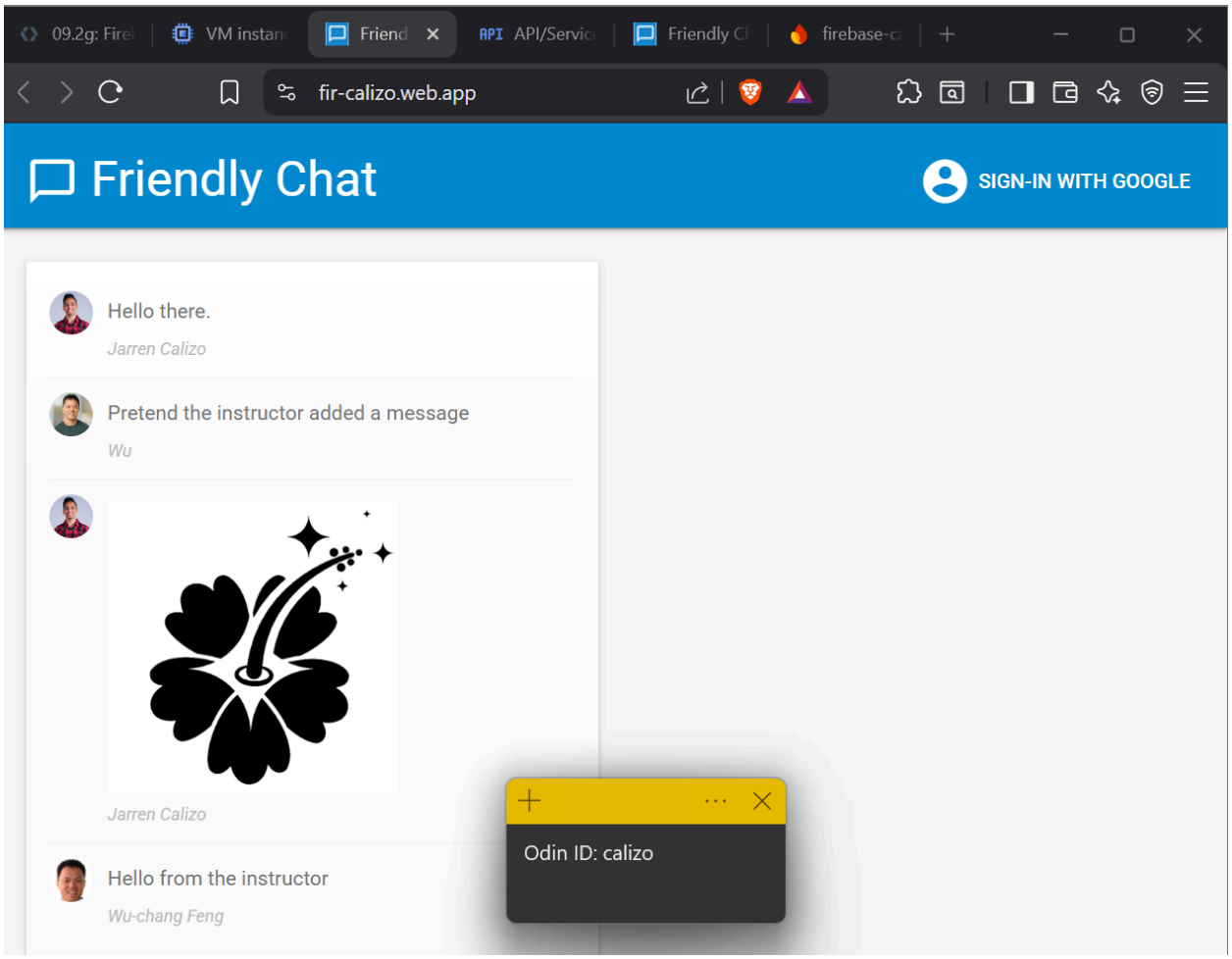
- ./public

I then deployed the app using:

firebase deploy --except functions

Firebase returned the production hosting URL ending in .web.app. Visiting that URL showed the deployed Friendly Chat app running exactly as the local emulator version.

2. Take a screenshot of the message including the URL for your lab notebook.



Step 21. Clean Up

To finish, I deleted the Firebase project from the Firebase console, confirmed deletion on the GCP project page, and ensured the entire test environment was wiped clean. The lab closes where it began: everything ephemeral, everything cleaned up, everything learned packed forward into the next project.

Lab 9.3g BigQuery, BigLake

BigQuery has always felt like one of those quiet engines powering the modern data world. In this lab, I finally got to watch it move from theory on a slide deck to something tactile: a dataset, a schema, a warehouse, and the lake that waits just outside of it. The steps below chronicle that exploration, grounded both in the native BigQuery storage layer and the external BigLake architecture sitting atop Cloud Storage. Every step pushed me a little closer to seeing why BigQuery feels effortless even at scale.

Step 1. Enabling BigQuery Services

I began by listing all available services matching "bigquery", then enabled both the BigQuery API and the BigQuery Connection API through Cloud Shell. This ensured the environment was ready for queries from both the web console and the bq CLI.

Step 2. Examining the Dataset

I downloaded the newborn-names CSV with wget, checked the header layout with head, and used wc -l to confirm the total count of name rows.

From inspection:

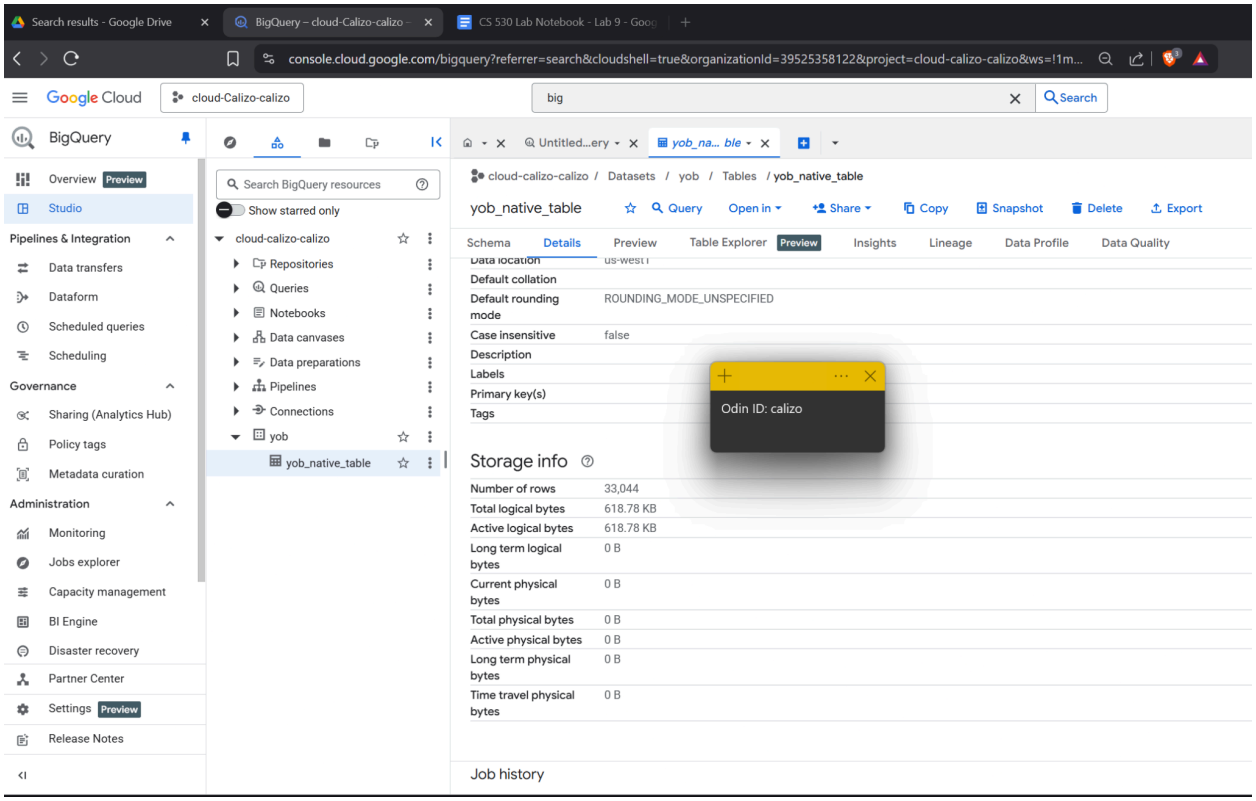
- name -> STRING
- gender -> STRING
- count -> INTEGER

I also downloaded the file locally for the upcoming upload step.

Step 3. Create dataset

Inside the BigQuery web console, I created a dataset named yob in region us-west1. Then I created a table by uploading the CSV file and manually defining the schema.

1. Take a screenshot of the table's details that includes the number of rows in the table.

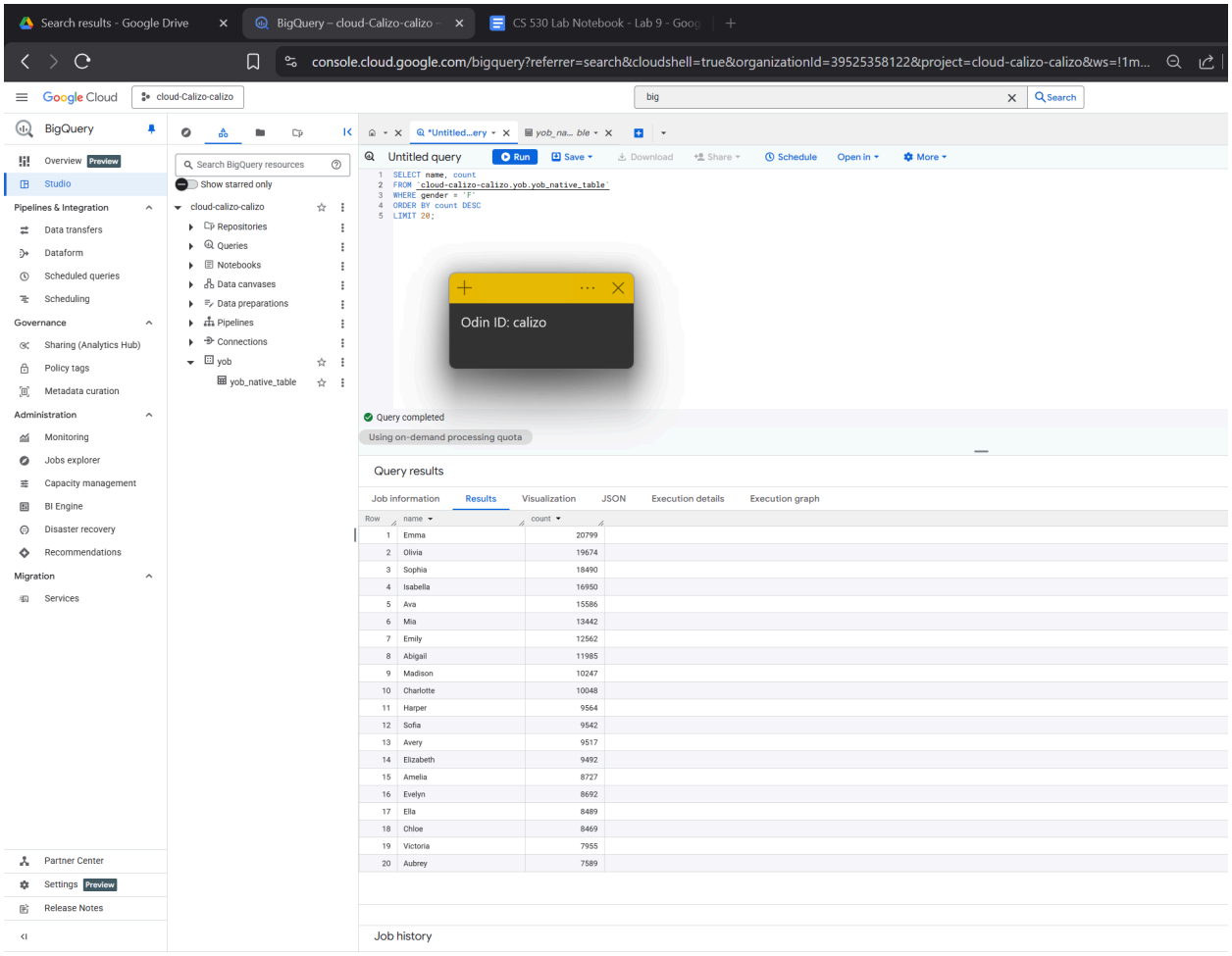


This shows yob_native_table with 33,044 rows, confirming the dataset imported correctly.

Step 4. Query data

Here, I moved through three different querying interfaces: the Web UI, the CLI, and the interactive bq shell. Each one reaffirmed why SQL remains one of the most universal languages in computing.

1. Screenshot the query results and include it in your lab notebook



The screenshot shows the top 20 female names sorted by descending count. The validator also confirmed the read size before execution, which the lab emphasizes for cost-awareness. In other words, it displays names like Emma, Olivia, Sophia, etc., in their correct order.

2. Screenshot your results and include it in your lab notebook



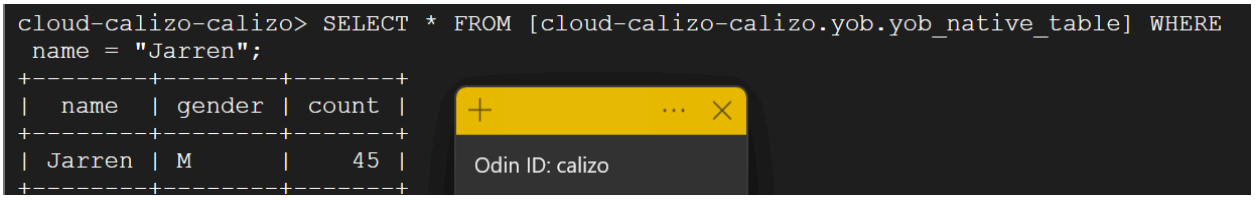
Using the bq command with standard SQL disabled (--use_legacy_sql=false), I queried the 10 least frequent male names. In other words, this clearly shows names with count = 5, confirming the correctness of the query.

3. Screenshot your results and include it in your lab notebook



From the interactive prompt, I queried the top 10 male names in 2014. In other words, this reflects the expected results (Noah, Liam, Mason, etc.) sorted by descending count.

4. Screenshot your results and include it in your lab notebook



I searched for "Jarren". BigQuery returned 45 entries for 2014. In other words, this shows the row for my name.

Step 5. Beginning the BigLake Component

This step bridged the conceptual shift between warehouse-style storage (BigQuery native tables) and a lake-style architecture where data remains in Cloud Storage and queries access it through connectors. No execution was needed here beyond ensuring understanding.

Step 6. Creating the External Table (BigLake)

In Cloud Shell, I:

1. Created a bucket in us-west1.
2. Uploaded the CSV file into that bucket.
3. Returned to BigQuery to create a second table, this time specifying:
    a. External table
    b. Source: GCS file
    c. Name: yob_biglake_table
    d. A new BigLake connection

Step 7. Configuring the BigLake Connection

Following the UI flow, I created a connection with ID biglake in region us-west1, then defined the same schema as in the native table. The table creation completed successfully.

Step 8. Granting Storage Permissions to the Connector

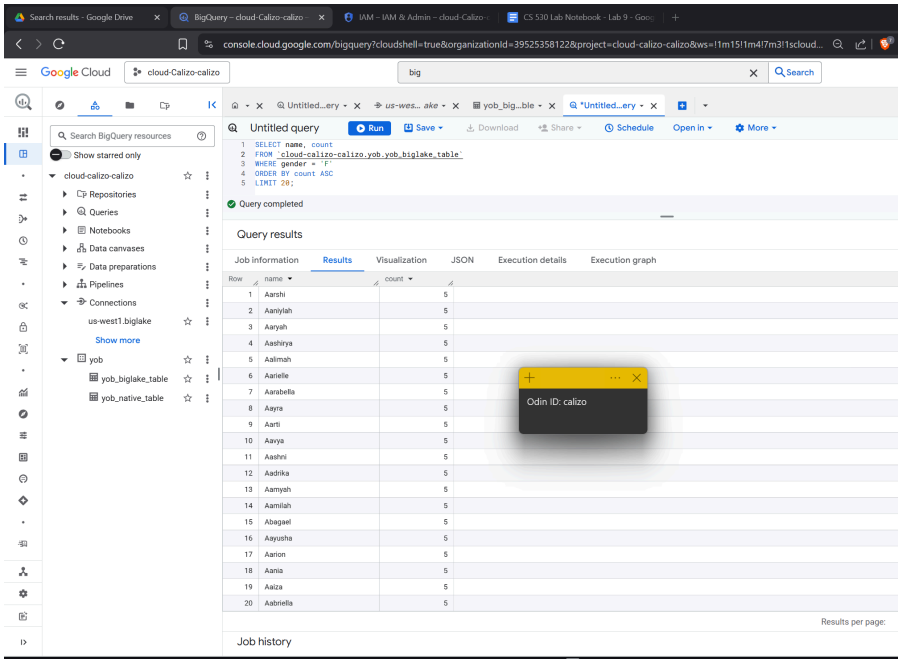
I located the service account for the BigLake connection under BigQuery > External connections. Then I granted it Storage Object Viewer permissions to the bucket

holding yob2014.csv. This step ensures the connector can read the file when queries run.

Step 9. Query data

With permissions set, I queried the external table for the 20 least popular female names.

1. Screenshot the query results and include it in your lab notebook



This shows 20 entries where each name has count = 5, matching expected results.

<u>Insight</u>

Walking through both native and external BigQuery tables felt like learning two dialects of the same language. One stores everything within the warehouse, optimized and polished. The other reaches outward, pulling insights directly from raw files living in the storage lake. Together, they reflect the dual nature of modern data platforms: structured when we need certainty, and fluid when we need flexibility. This lab made that duality feel real.

Lab 9.4g Jupyter Notebooks

Step 1. Notebooks Lab 1 (Natality)

I began by enabling all required Vertex AI and compute-related APIs listed in the instructions. With those active, I created a dedicated service account named cs430jupyter and granted it the roles/bigquery.user permission.

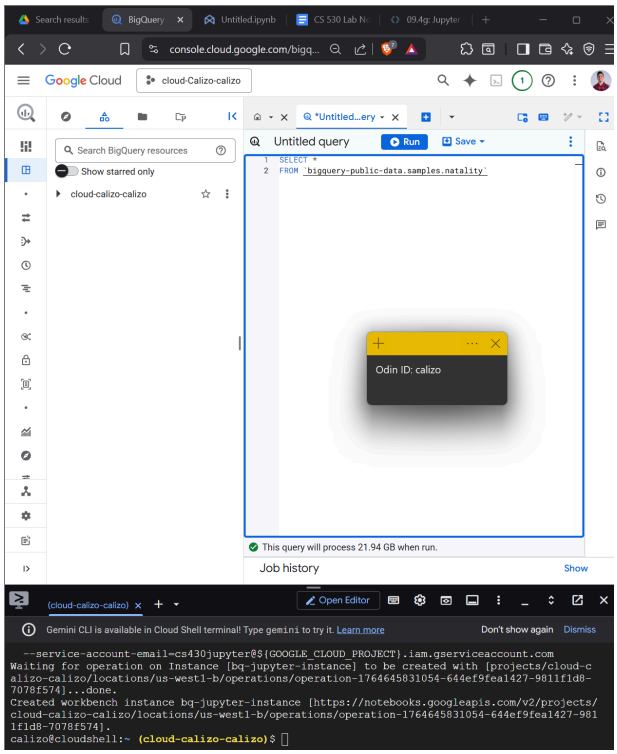
Afterward, I launched my Vertex AI Workbench instance using the provided gcloud workbench instances create command. The instance came online successfully, as reflected in the Cloud Shell output.

Step 2. Launch notebook

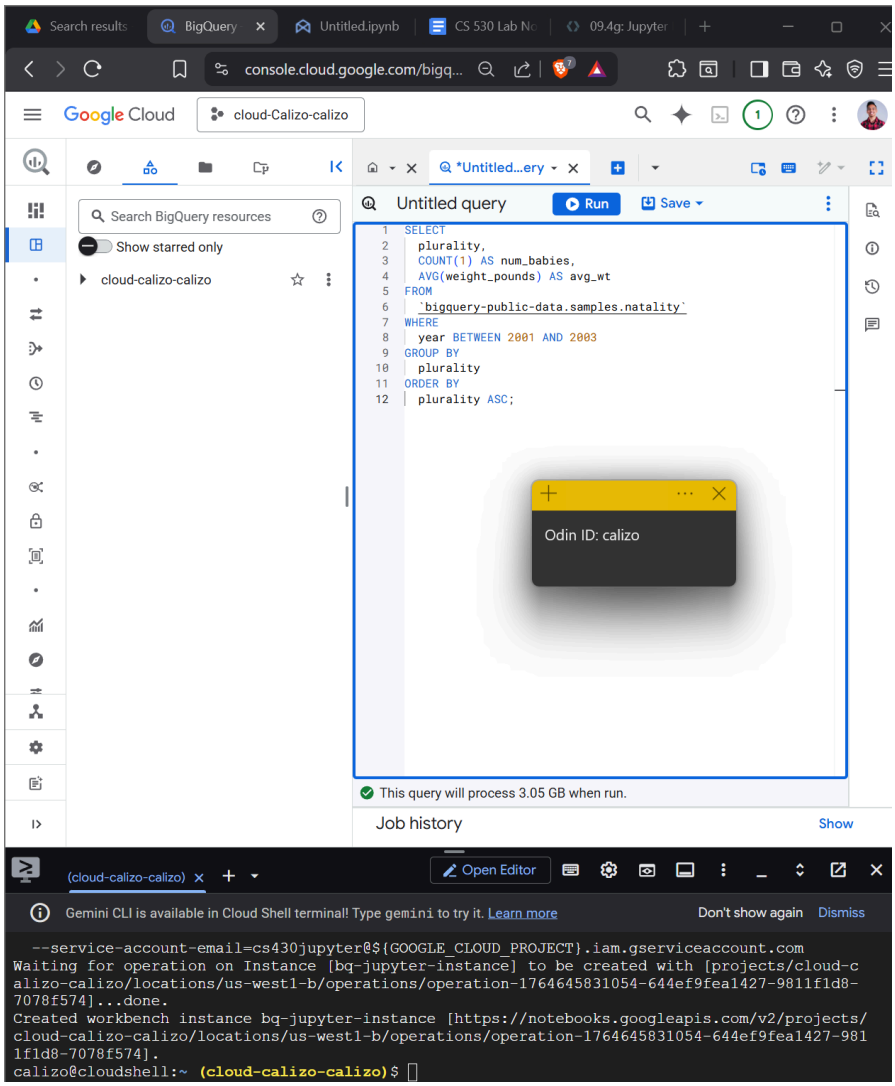
I opened Vertex AI in the console, navigated to Workbench, and launched JupyterLab on the instance. A new Python 3 notebook was created and left ready for BigQuery integration.

Step 3. BigQuery query

Before running any analytics, I queried the full natality table only to inspect its size. BigQuery estimated that a full table scan would process 21.94 GB.



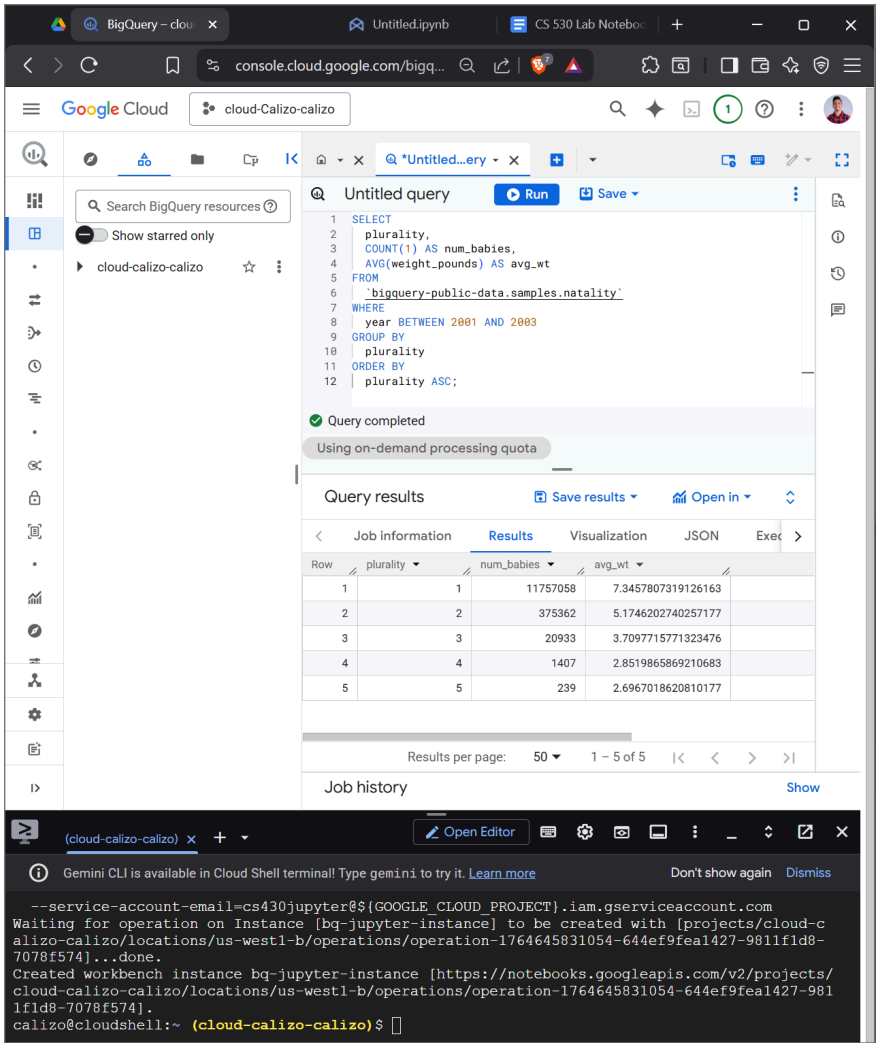
After modifying the query to filter for babies born between 2001 and 2003, the estimator showed approximately 3.05 GB.



1. How much less data does this query process compared to the size of the table?

The optimized query processed about 18.89 GB less data than a full-table scan.

2. How many twins were born during this time range?



The query output (screenshotted) shows the **plurality = 2** row with the number of babies.

- There were 375,362 twins born between 2001 and 2003.

3. How much lighter on average are they compared to single babies?

From the results:

- Single births (plurality 1) average weight: ~7.35 pounds
- Twins (plurality 2) average weight: ~5.17 pounds
    - Difference: approximately 2.18 pounds lighter.

Twins were about 2.18 pounds lighter on average than single babies.

Step 4. Jupyter notebook query

Inside the notebook, I constructed the same natality query in a triple-quoted Python string and used the BigQuery Python client to fetch a DataFrame. The results aligned with the values from the BigQuery console.

I then generated a scatter plot of plurality vs average birth weight.
The plotted curve clearly reflected the downward slope from singletons to higher-order multiples.

Step 5. Exploring the dataset

To look beyond plurality, I ran the exploratory query gathering weight, gender, mother_age, plurality, and gestation weeks for babies born after 2000. The helper function get_distinct_values(column_name) enabled quick analysis of how each variable correlates to newborn weight.

The preliminary DataFrame output confirmed correct ingestion.

Step 6. Run queries

I generated four bar plots showing average birthweight as a function of:

- plurality
- is_male
- gestation_weeks
- mother_age

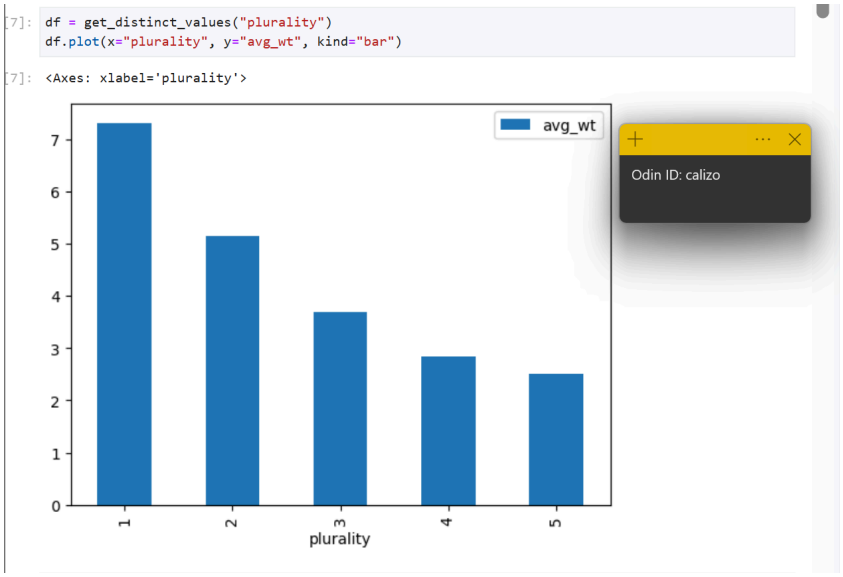
I examined the clear patterns:

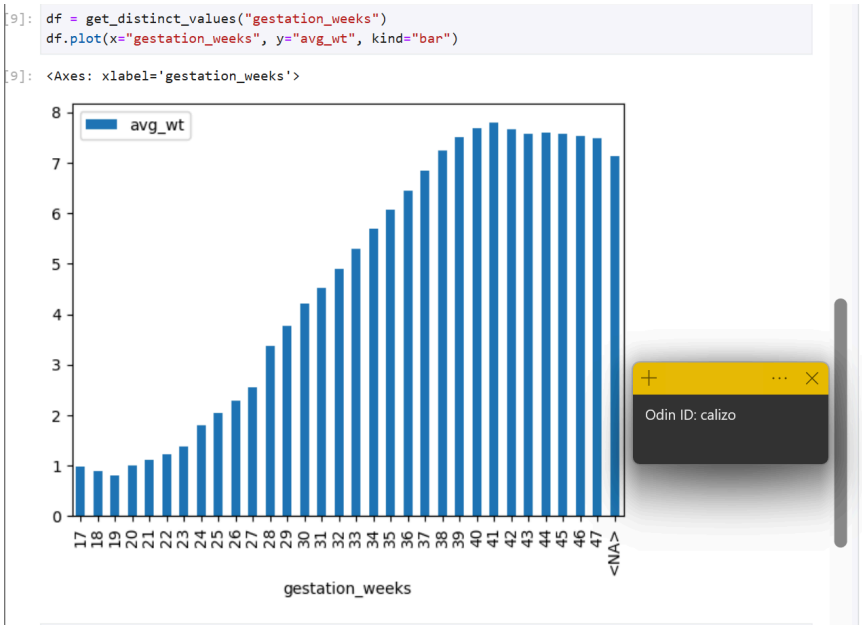
- plurality exhibits a pronounced decline in average weight as the number of babies increases.
- gestation_weeks rises sharply and steadily, peaking in the 38 to 41 week range.
- mother_age follows a subtle curve but with far less dramatic impact and overall subtle relationship compared to plurality and gestation weeks.
- is_male shows only a small difference between male and female averages, not nearly as strong.

1. Show the plots generated for the two most important features for your lab notebook

The two features that created the strongest, visually obvious predictive signals were:

1. Gestation weeks
    a. The bar chart exhibits a near-linear upward weight progression with gestational length.
    b. This was the strongest and most physiologically intuitive predictor.
2. Plurality
    a. The weight drop from single to multiple births is immediate and steep.
    b. This feature is both categorical and biologically impactful.





Step 7. Notebooks Lab 2 (COVID-19 data)

I added the BigQuery public datasets and starred the COVID-19 dataset. I then inspected the mobility dataset covid19_google_mobility.mobility_report.

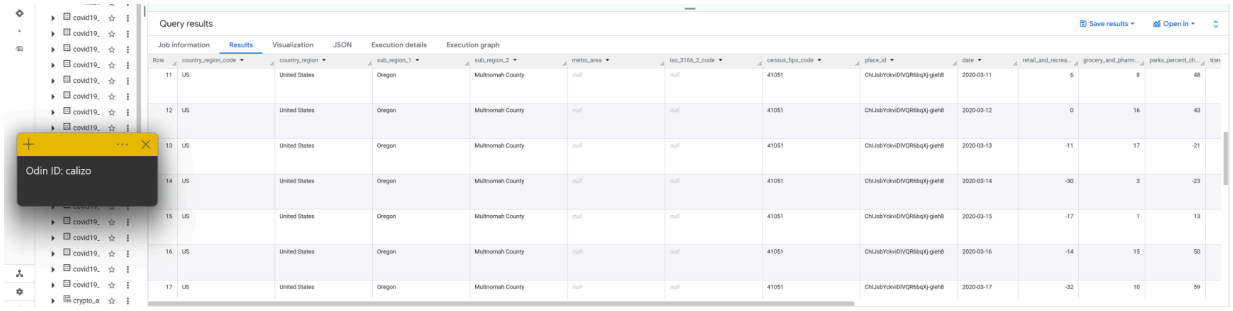
Step 8. Mobility

I ran the provided query filtering for March 2020.

1. What day saw the largest spike in trips to grocery and pharmacy stores?

Using the table, I scanned the grocery_and_pharmacy_percent_change_from_baseline column.

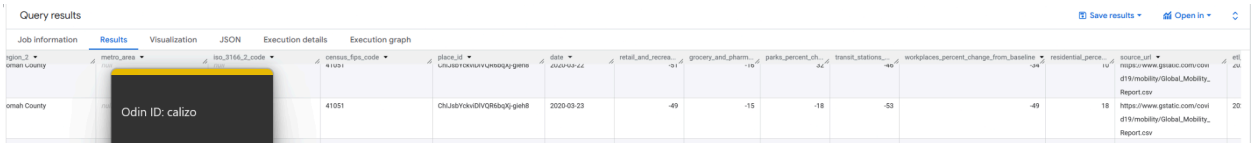
This shows the strongest positive spike during March to be:



- March 13, 2020 had the largest spike in grocery and pharmacy trips.

2. On the day the stay-at-home order took effect (3/23/2020), what was the total impact on workplace trips?

On that date, the workplaces_percent_change_from_baseline column shows:

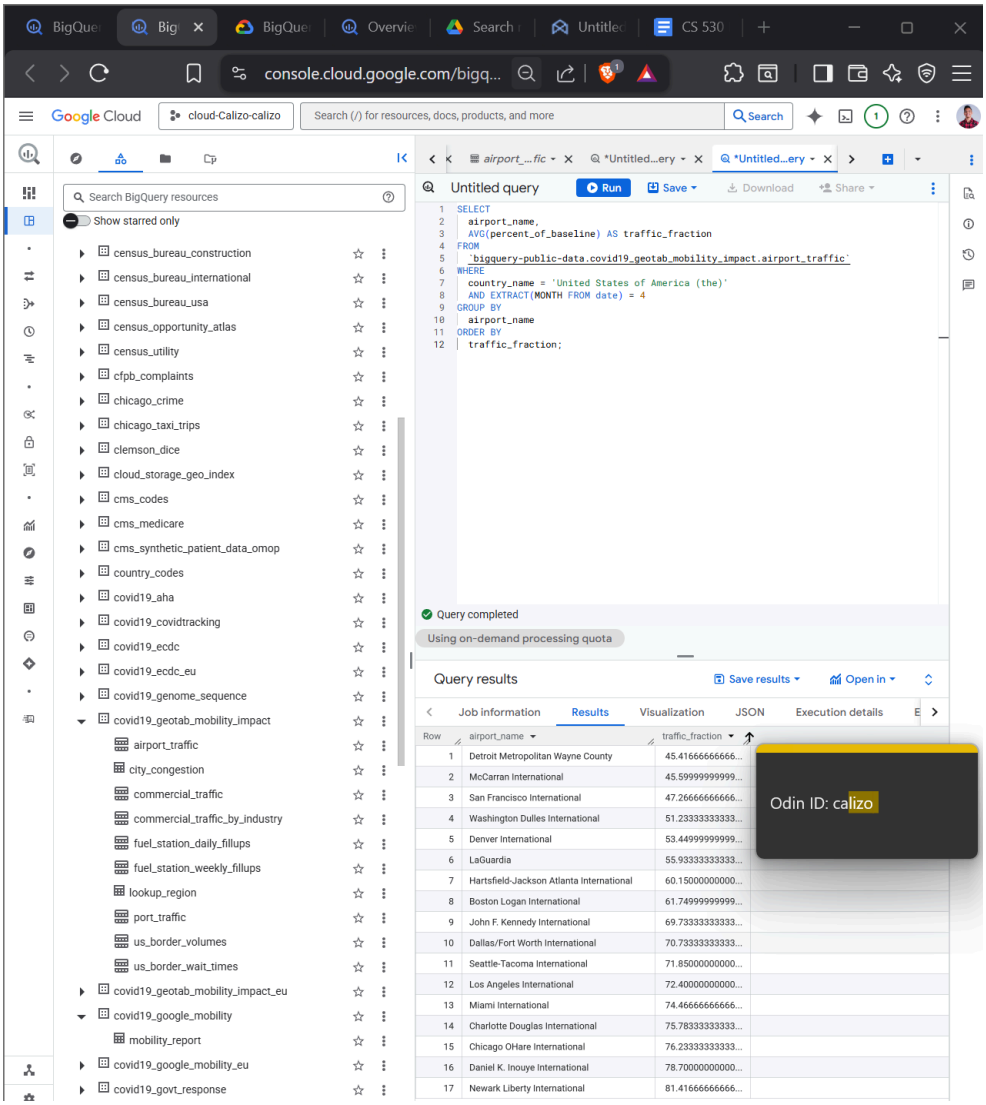


- Workplace mobility decreased by approximately 53 percent on 3/23/2020.

Step 9. Airport traffic

Using the provided template, I computed AVG(percent_of_baseline) for all airports in April 2020.

1. Which three airports were impacted the most in April 2020 (the month when lockdowns became widespread)?



From these query results (sorted ascending by traffic_fraction):

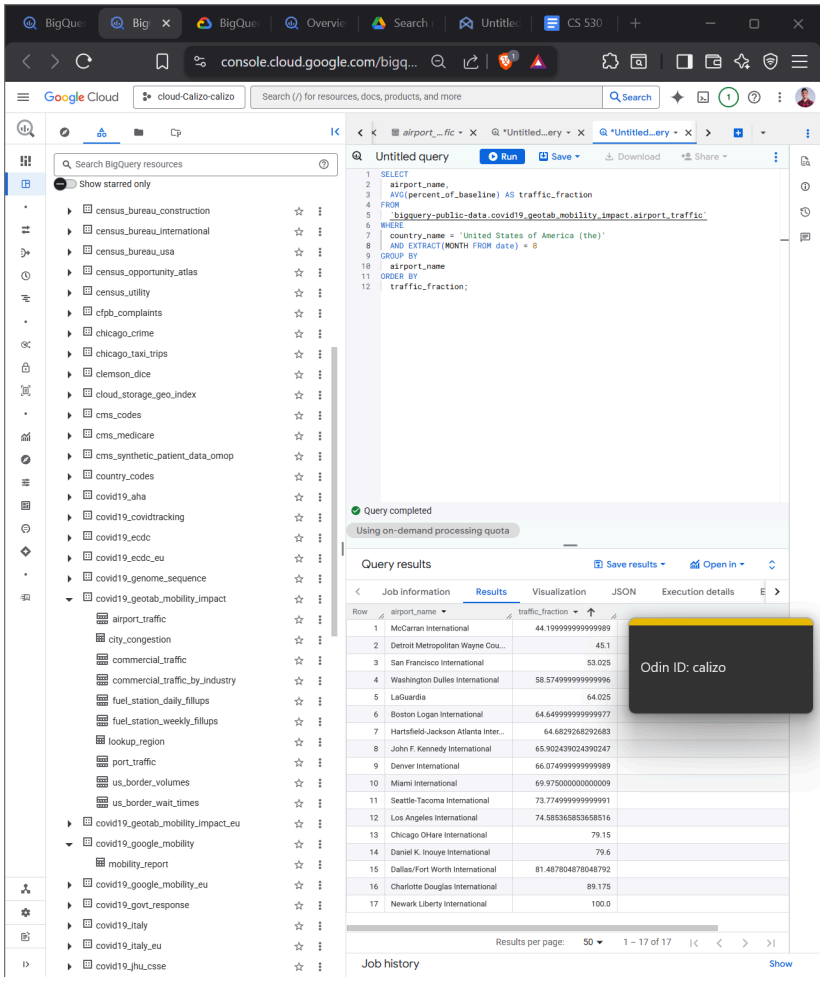
1. Detroit Metropolitan Wayne County
2. McCarran International
3. San Francisco International

The three most impacted airports in April 2020 were Detroit, McCarran, and San Francisco.

2. Run the query again using the month of August 2020. Which three airports were impacted the most?

From this (August results):

1. McCarran International
2. Detroit Metropolitan Wayne County
3. San Francisco International



- The most impacted airports in August 2020 were again McCarran, Detroit, and San Francisco.

<u>Insight</u>

This lab stitched together the analytical strengths of BigQuery with the narrative finesse of Python in a notebook. Visualizing plurality and gestation effects made the natality dataset feel alive, while revisiting early-COVID mobility shifts was a sobering reminder of how suddenly patterns can change. Across both labs, the practice of reducing massive public datasets into meaningful slices felt both empowering and humbling.

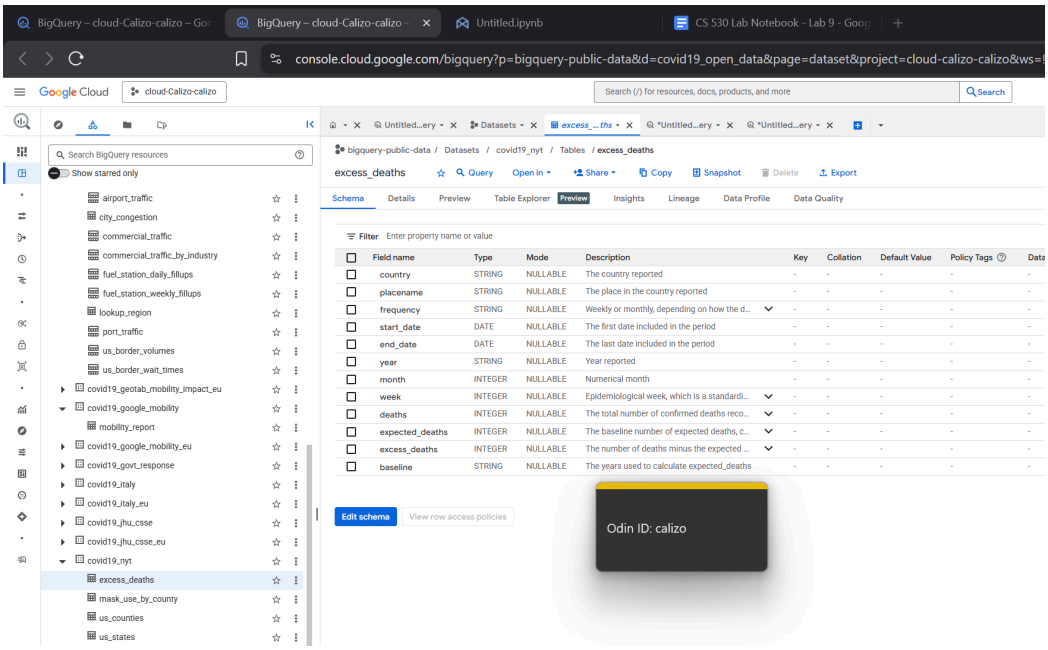
Even as datasets shift from birthweight to pandemic curves, the narrative of this lab stayed the same for me: using data to reclaim clarity. Query by query, I watched the story of COVID-19 unfold inside BigQuery and Jupyter, where each table became a window into real human behavior, loss, and resilience. These last four steps brought the emotional weight of the dataset to the foreground, reminding me how data science is not just computation but memory.

Step 10. Mortality

Stepping into the New York Times COVID-19 dataset felt like returning to early-pandemic data journalism, navigating through tables that once shaped national understanding of risk. I inspected each schema directly in BigQuery, focusing on the fields that tie geography, time, and health outcomes together.

I began by exploring the schema of the four New York Times COVID-19 tables inside bigquery-public-data.covid19_nyt. The screenshots reflect this schema analysis.

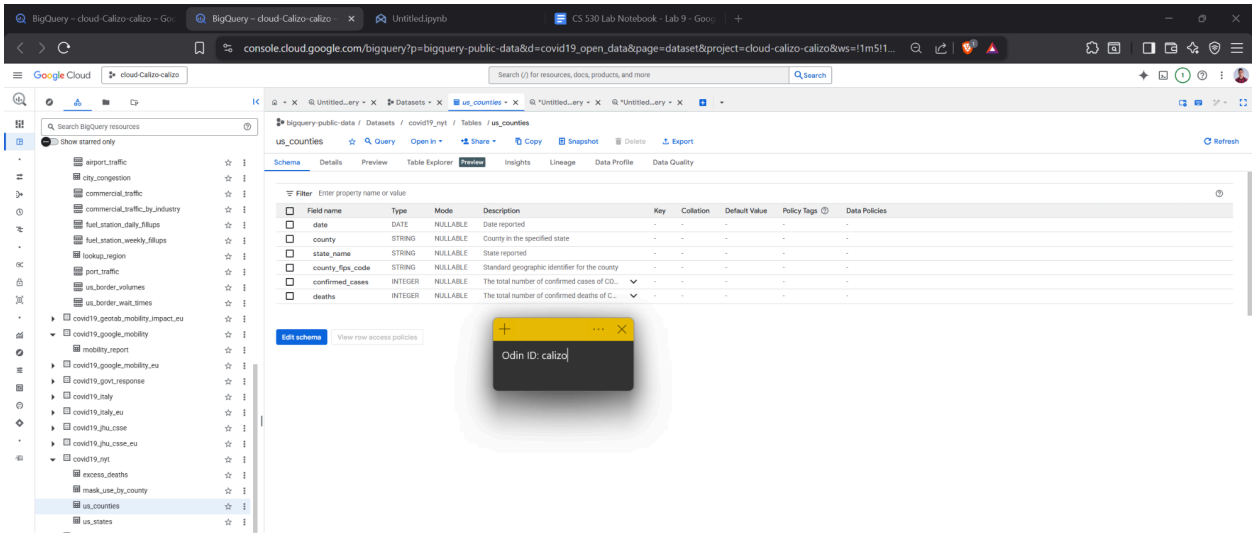
1. What table and columns identify the place name, the starting date, and the number of excess deaths from COVID-19?



From the excess_deaths table, the fields that correspond to:

- Placename: placename
- Starting date: start_date
- Excess deaths: excess_deaths

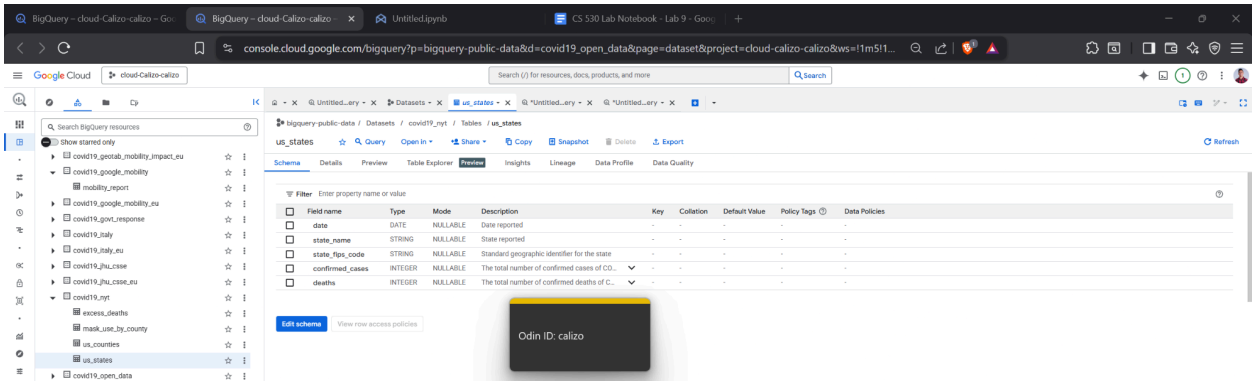
2. What table and columns identify the date, county, and deaths from COVID-19?



From us_counties:

- Date: date
- County: county
- Deaths: deaths

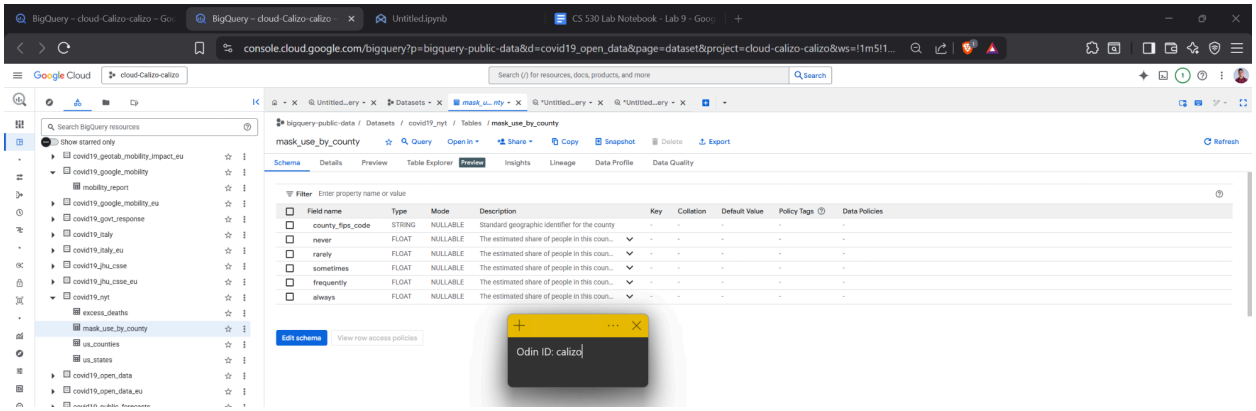
3. What table and columns identify the date, state, and confirmed cases of COVID-19?



From us_states:

- Date: date
- State: state_name
- Confirmed cases: confirmed_cases

4. What table and columns identify a county code and the percentage of its residents that report they always wear masks?



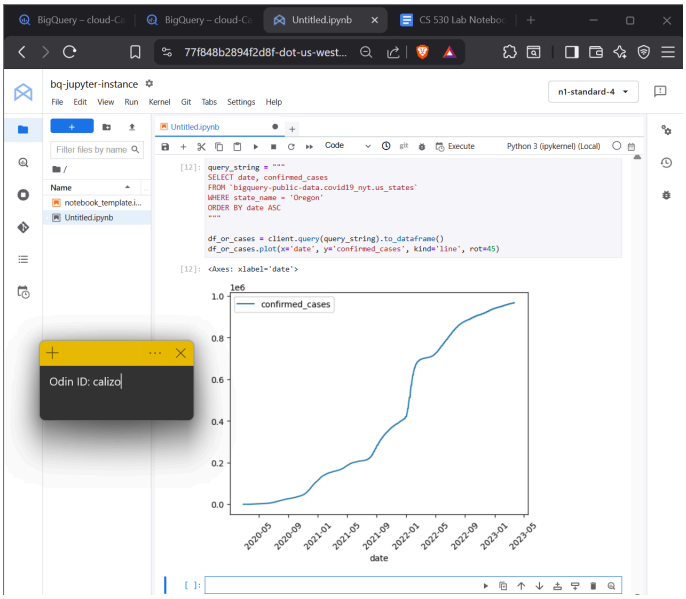
From mask_use_by_county:

- County code: county_fips_code
- Always wear mask percentage: always

Step 11. Run example queries

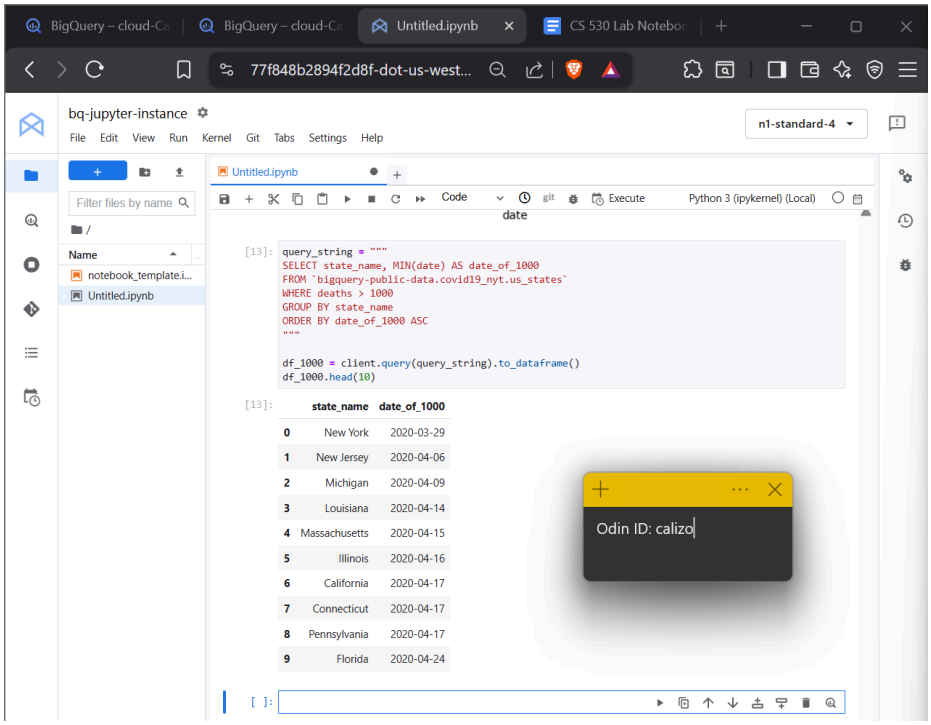
I ran the provided query inside the Jupyter notebook and used Pandas to visualize confirmed cases across Oregon.

1. Show a screenshot of the plot and the code used to generate it for your lab notebook



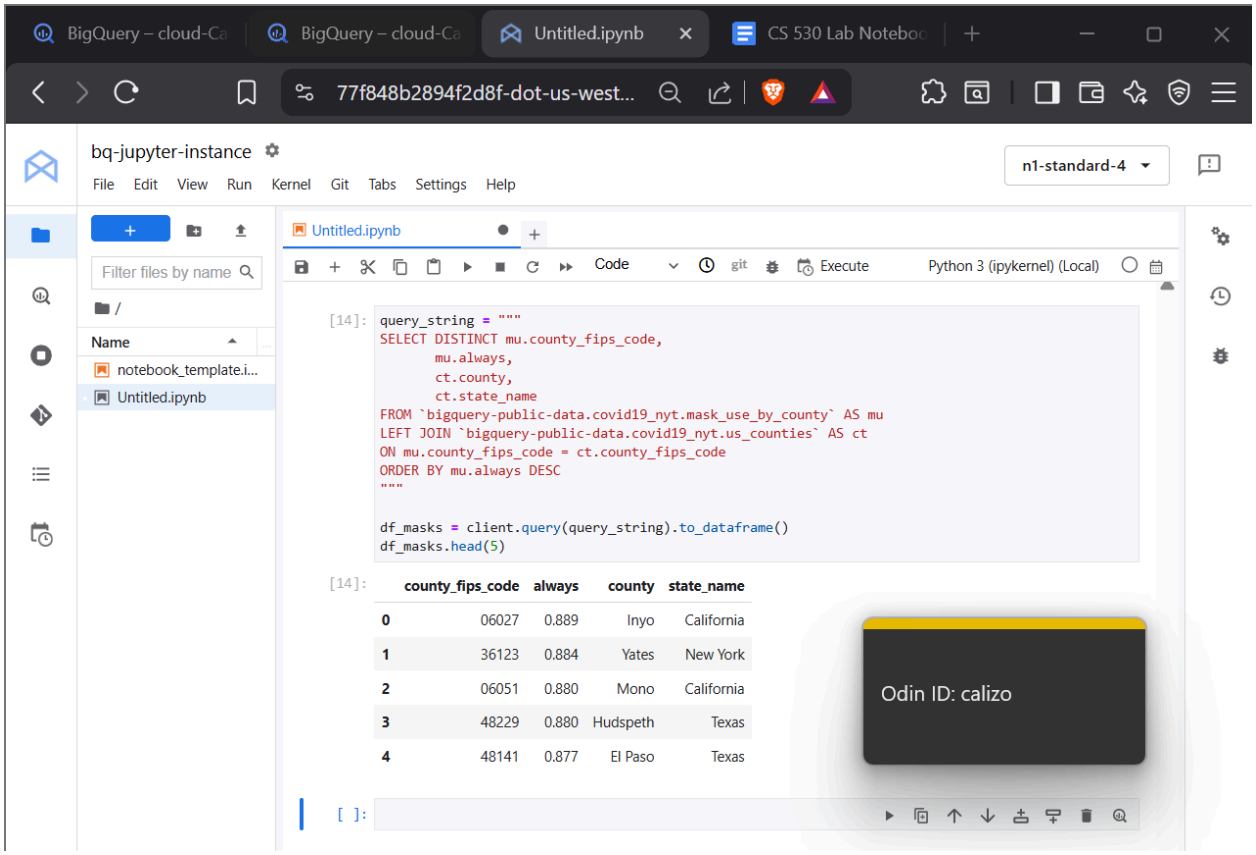
A screenshot of my plot and code is shown. The curve climbs sharply in 2022, matching the screenshot.

2. From within your Jupyter notebook, run the query and write code that shows the first 10 states that reached 1000 deaths from COVID-19. Take a screenshot for your lab notebook.



The query provided in the lab was executed inside the notebook. The screenshot shows the first 10 states that crossed 1000 cumulative COVID-19 deaths. New York and New Jersey stand at the top of the timeline.

3. Take a screenshot for your lab notebook of the Top 5 counties and the states they are located in.



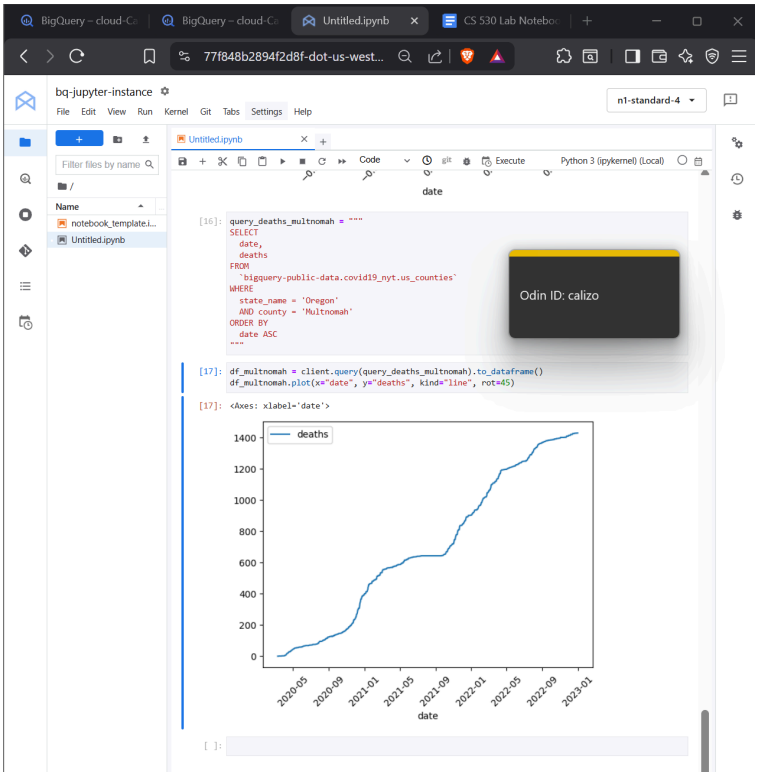
I ran the join between mask_use_by_county and us_counties. The screenshot reflects the Top 5 counties ranked by mask usage, with their state names included.

Step 12. Write queries

1. Plot the results and take a screenshot for your lab notebook.

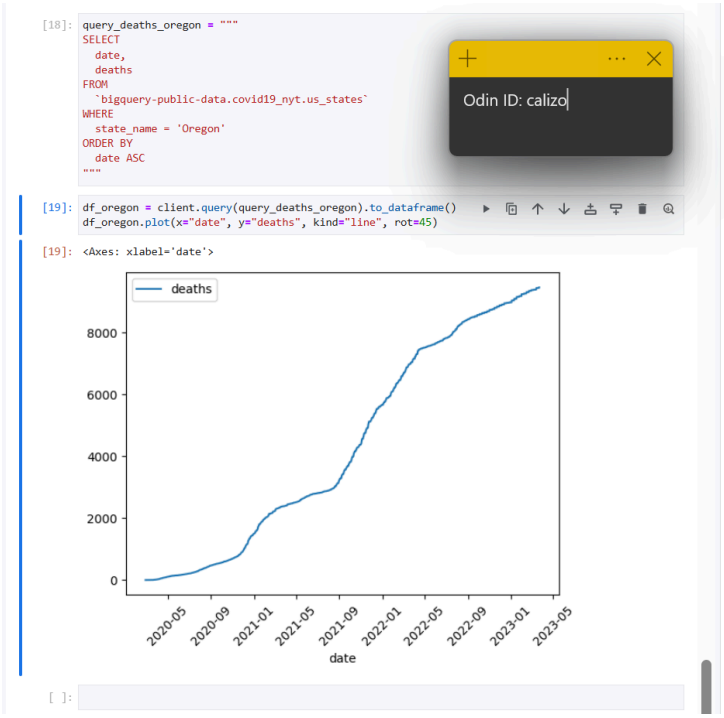
I constructed a query string in Jupyter that selected date and deaths from us_counties where:

state_name = 'Oregon'
AND county = 'Multnomah'
ORDER BY date ASC



The plot in the screenshot shows a steady rise across the pandemic timeline.

2. Plot the results and take a screenshot for your lab notebook.



I created another query to gather all Oregon deaths from the us_states table, ordered by date ascending. The statewide trajectory rises sharply in mid-2021, aligning with the screenshot.

Step 13. Cleanup

To close the lab, I tore down the notebook VM using:

```
gcloud workbench instances delete bq-jupyter-instance \
  --location us-west1-b
```

A fitting ending to a lab that began with birthweights and ended with mortality numbers - a reminder of how these datasets hold both technical insight and human story.